

Cloud Computing Technologies

DAT515 - Fall 2025

Docker

Prof. Hein Meling





What is Docker?

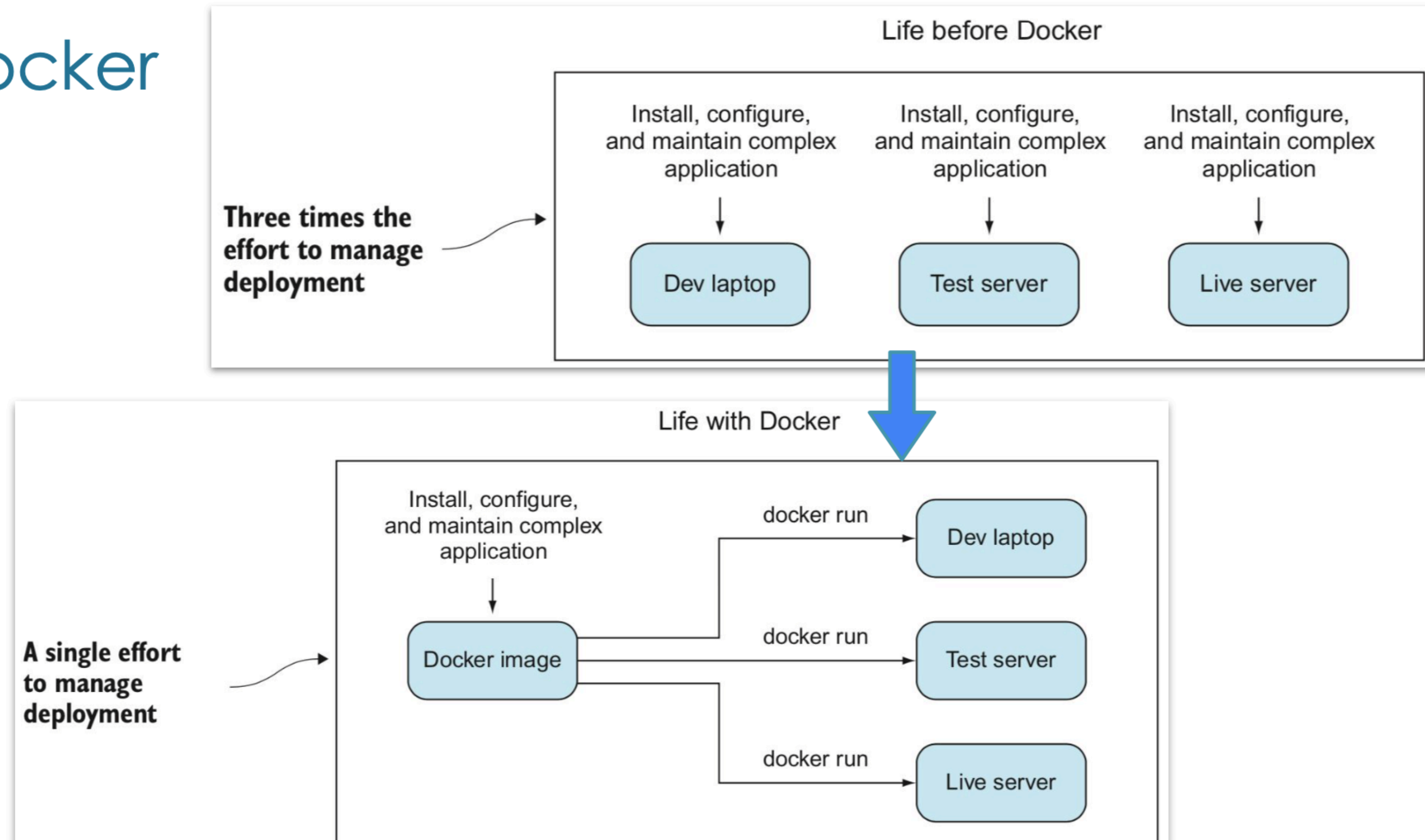


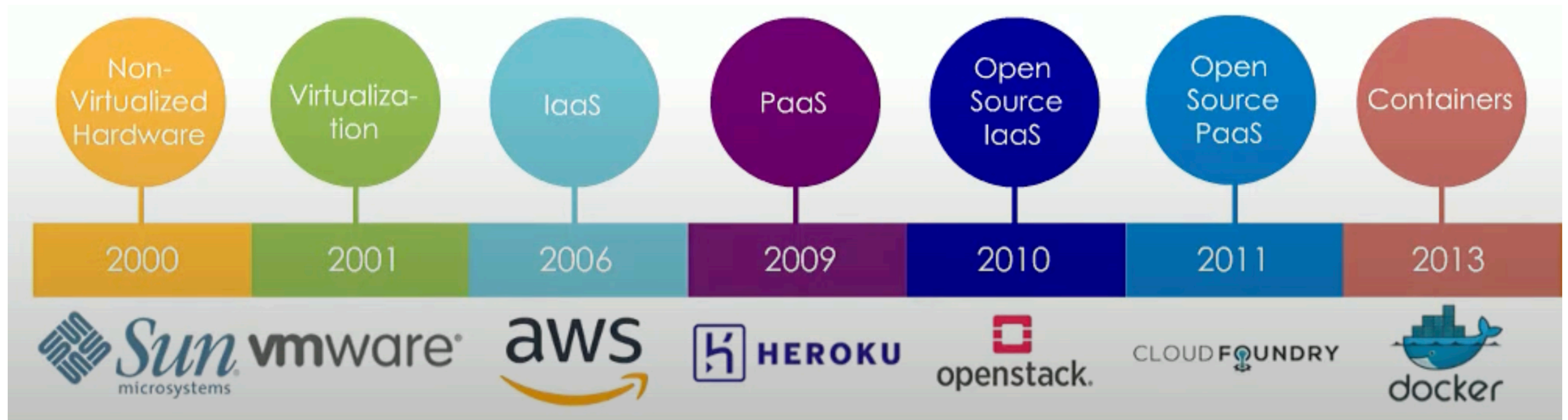
- Docker was introduced by Solomon Hykes in 2013
- Initially, seen as yet another virtualization tool
- But Docker is a containerization technology
- Revolutionized development and deployment workflows
- Automating and streamlining the deployment process

The word “Docker” comes from a British colloquialism meaning dock worker — somebody who loads and unloads ships.

- Docker provides an abstraction to simplify development and deployment
- Cross platform and open source
 - Runs on Windows, macOS, and Linux...
 - Runs on several processor architectures (amd64, arm64, ...)
- Community and company adoption
 - Amazon, Microsoft, and Google

Docker





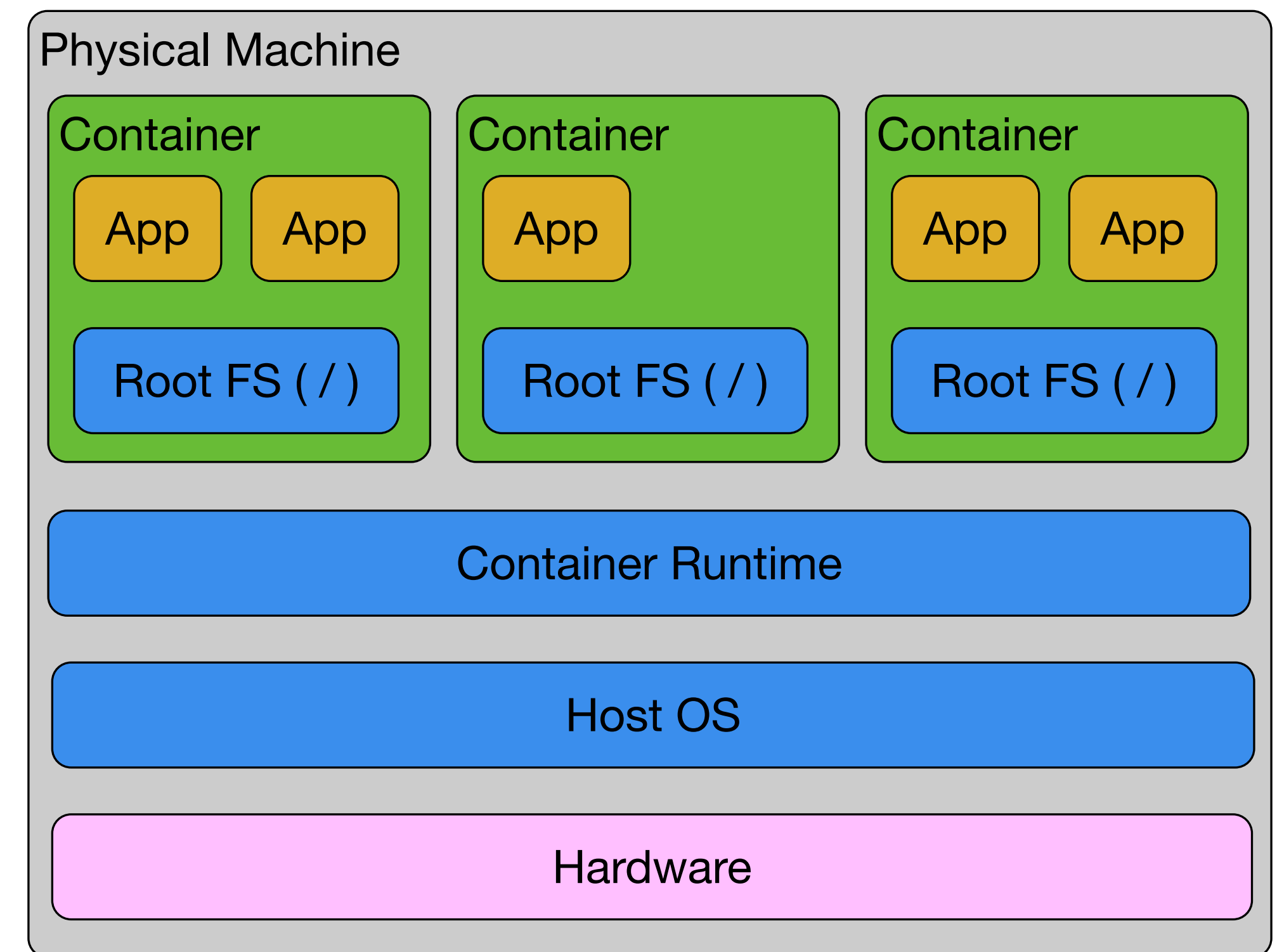
Docker has become
synonymous with Containers

What is Docker, Technically?

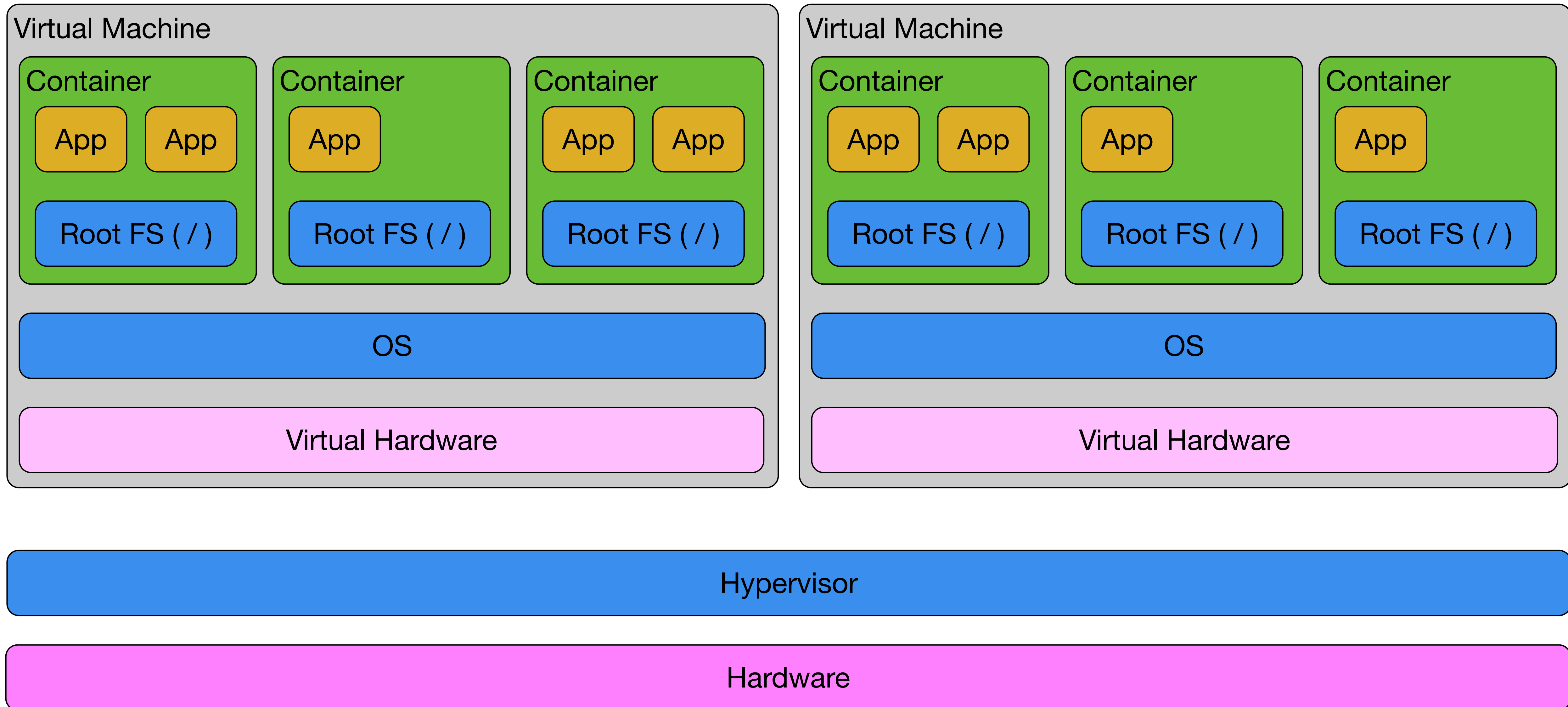
Lightweight, standalone, and executable software package

- Application code, Configuration files, Runtime libraries, Dependencies
- Pre-configured runtime environment
- Executable with a single command

Write once, run anywhere!

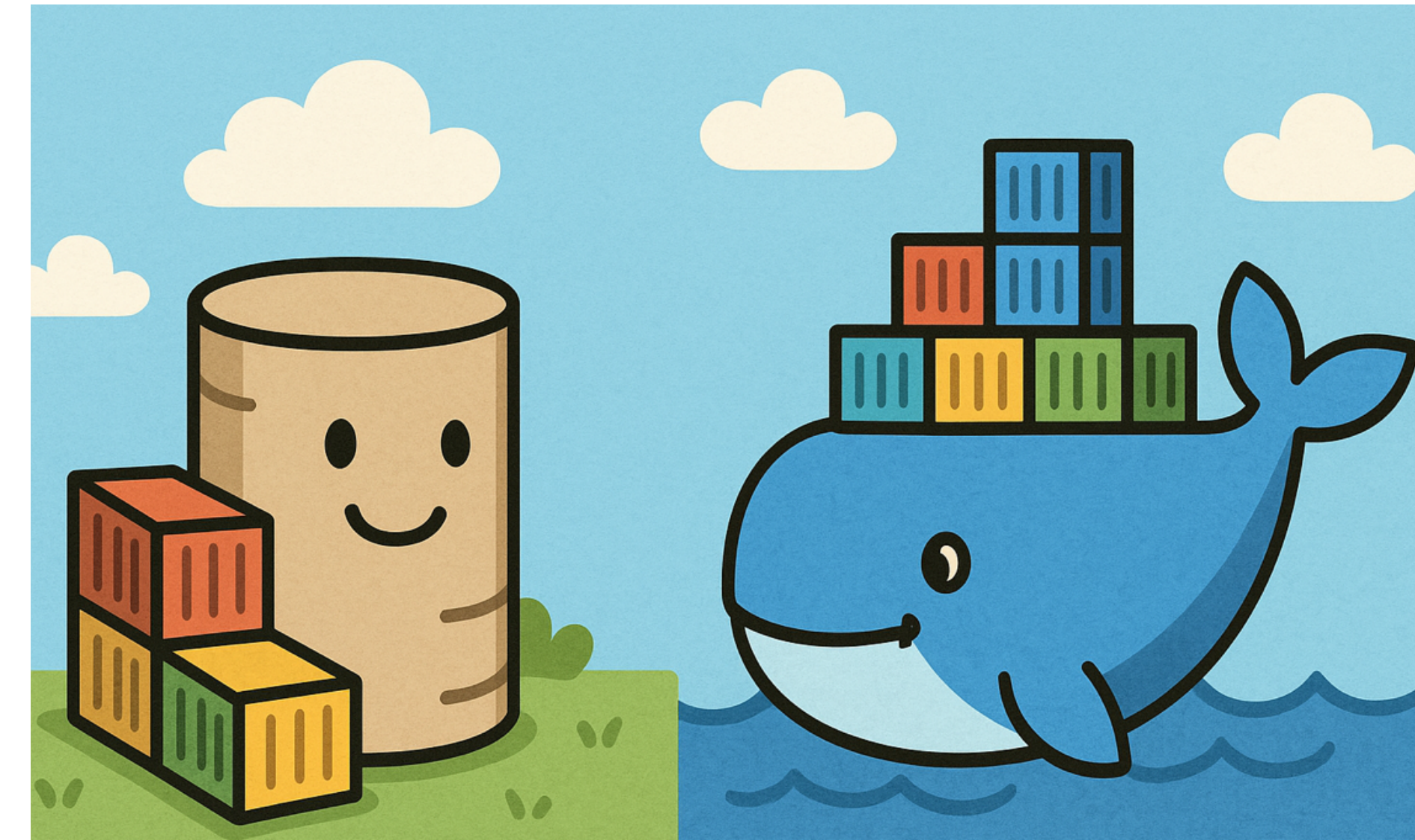


Containers inside Virtual Machines



What's the Difference?

- **Containers:** lightweight packaging of an application and its dependencies
 - Can run consistently across different environments
 - Share the host system's kernel, making them more efficient
- **Docker** is a platform and set of tools
 - Simplify the creation, deployment, and management of containers
 - Easy-to-use interface, standardized container format
 - Comprehensive ecosystem for building, distributing, and running containers



What's the Difference?

- Docker is based on the **runC** container runtime
 - Portable and compliant with the Open Container Initiative (OCI) standards
- LXC is tightly coupled to Linux environments (not portable)
- Docker is more than LXC
 - Command line tools and Desktop Client

docker desktop PERSONAL

Search for images, containers, volu... K

M

Settings [Give feedback](#)

Resources

Docker Engine

Builders

Kubernetes

Software updates

Extensions

Features in development

Notifications

Advanced

Selected builder

The default builder used when you start a build. [Learn more](#)

desktop-linux
docker

Running

Available builders

Inspect and manage your builders. [Learn more](#)

default
docker

Running

Cancel

Apply & restart

Engine running

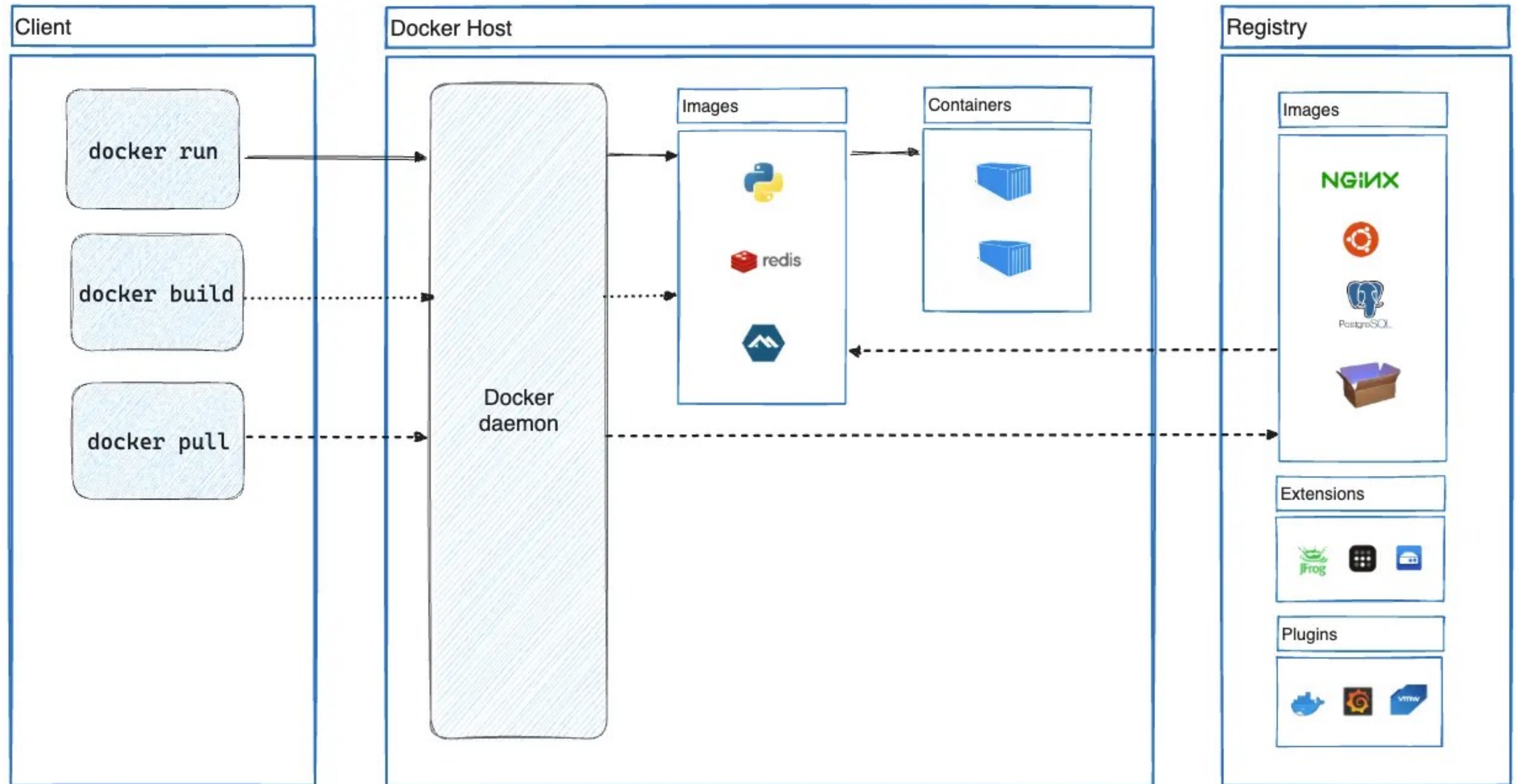
RAM 2.39 GB CPU 1.50% Disk 56.01 GB avail. of 62.67 GB

v4.33.0 2

Prof. Hein Meling

CLOUD COMPUTING

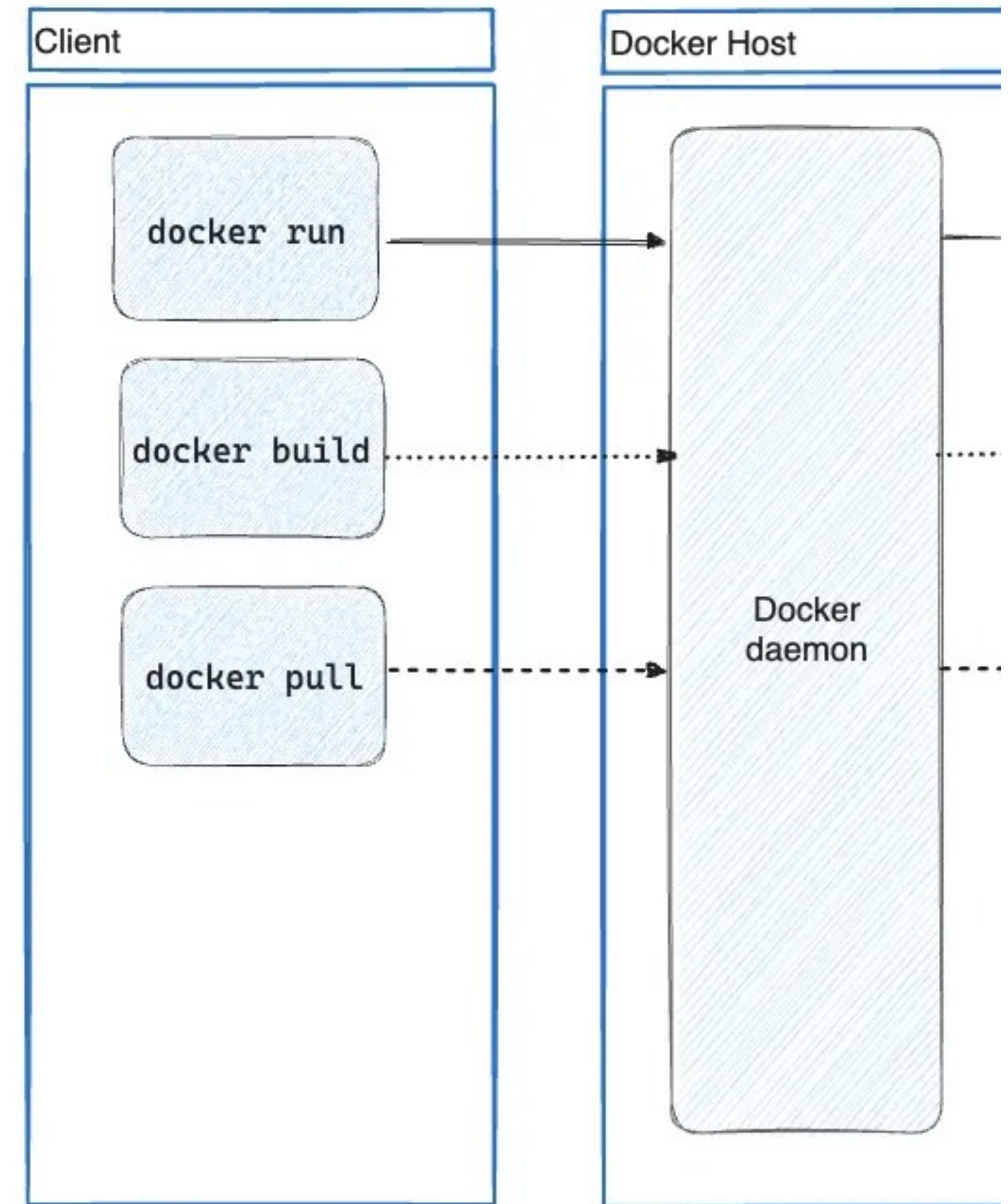
DOCKER



Docker Components

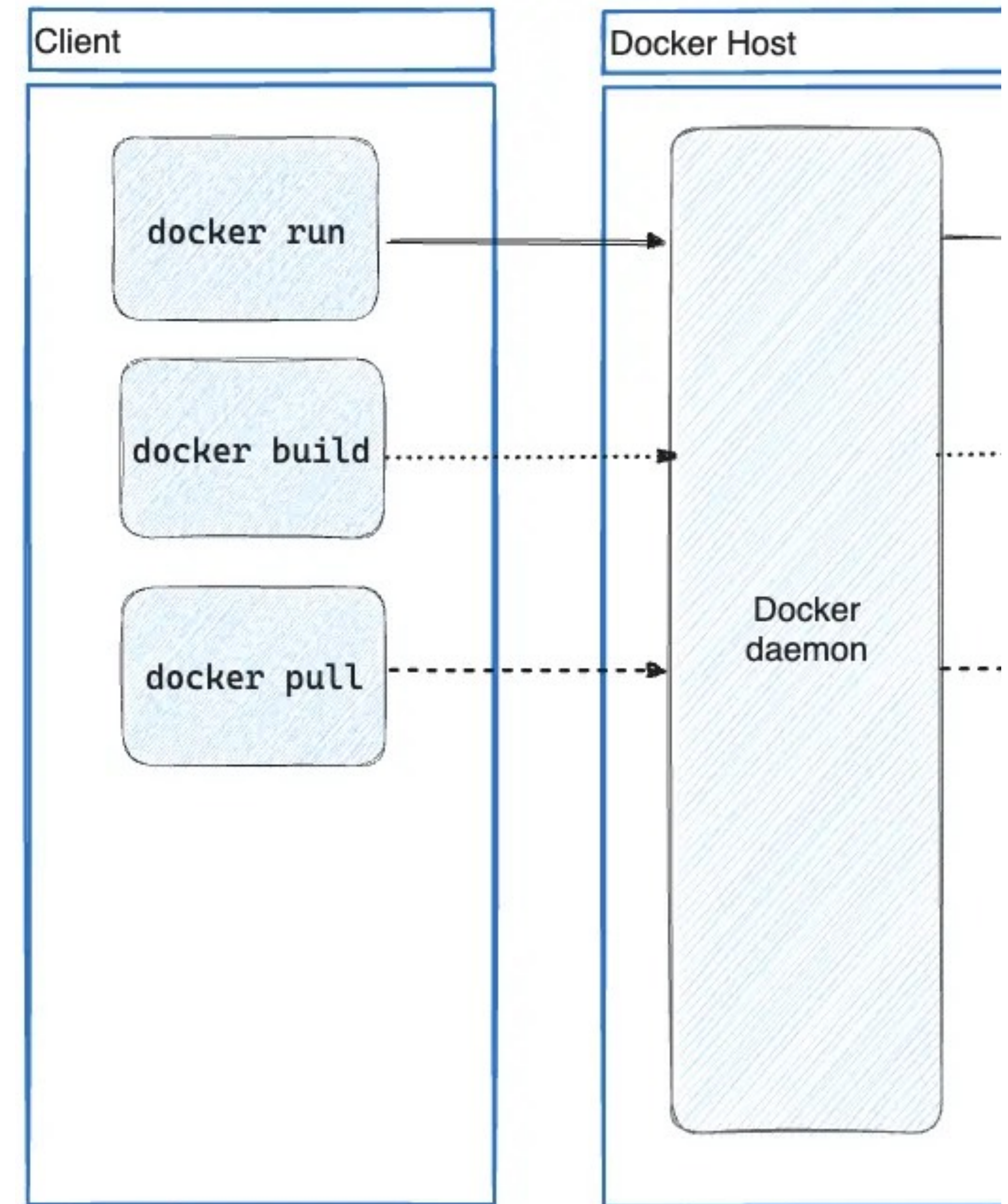
The Docker Daemon (Server)

- Runs and manages containers on a host system
 - Including the container runtime
- Client-Server Architecture
 - Clients sends commands to the daemon which handles container operations
- Supports building, running, and managing containers



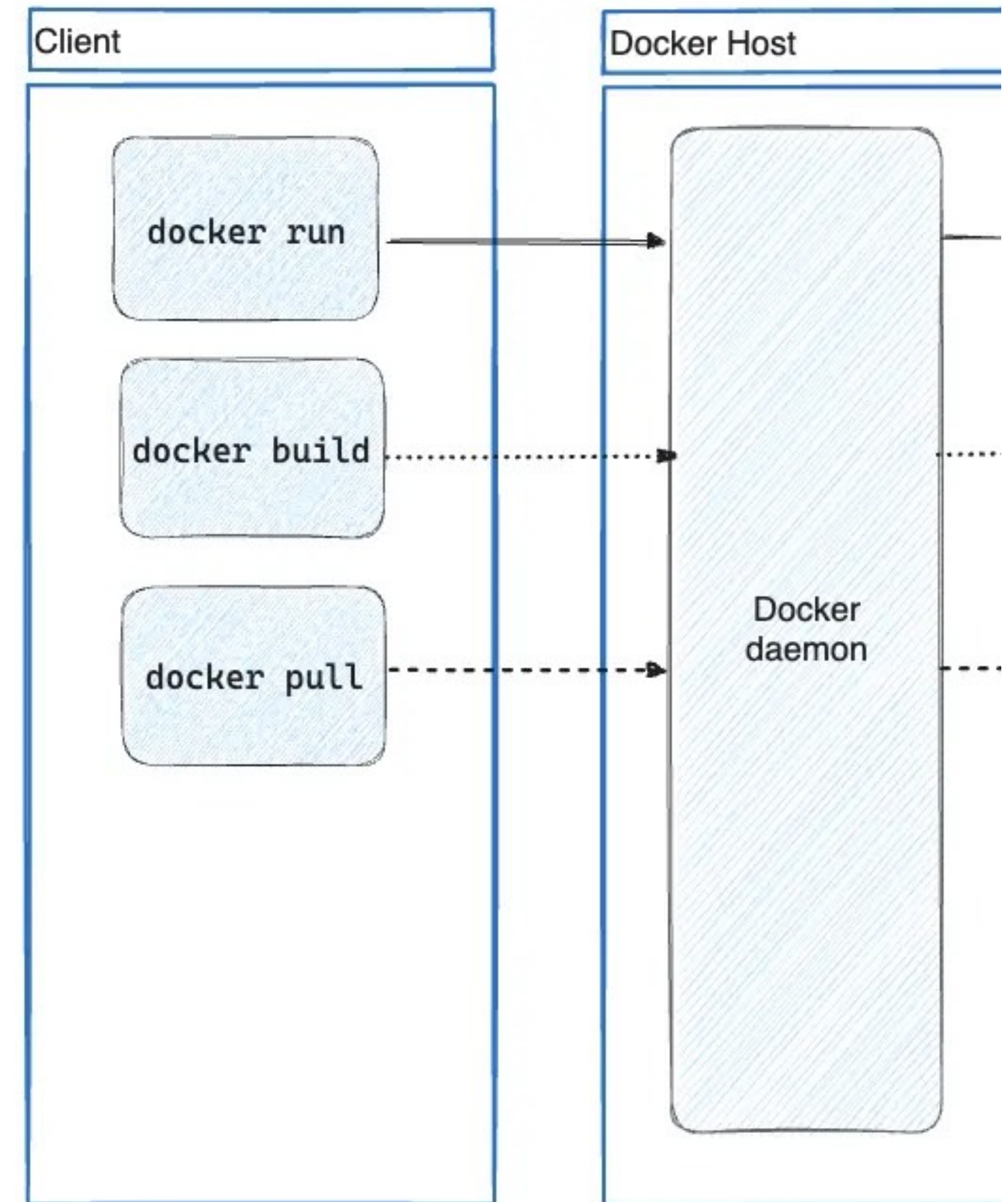
The Docker Command Line Client

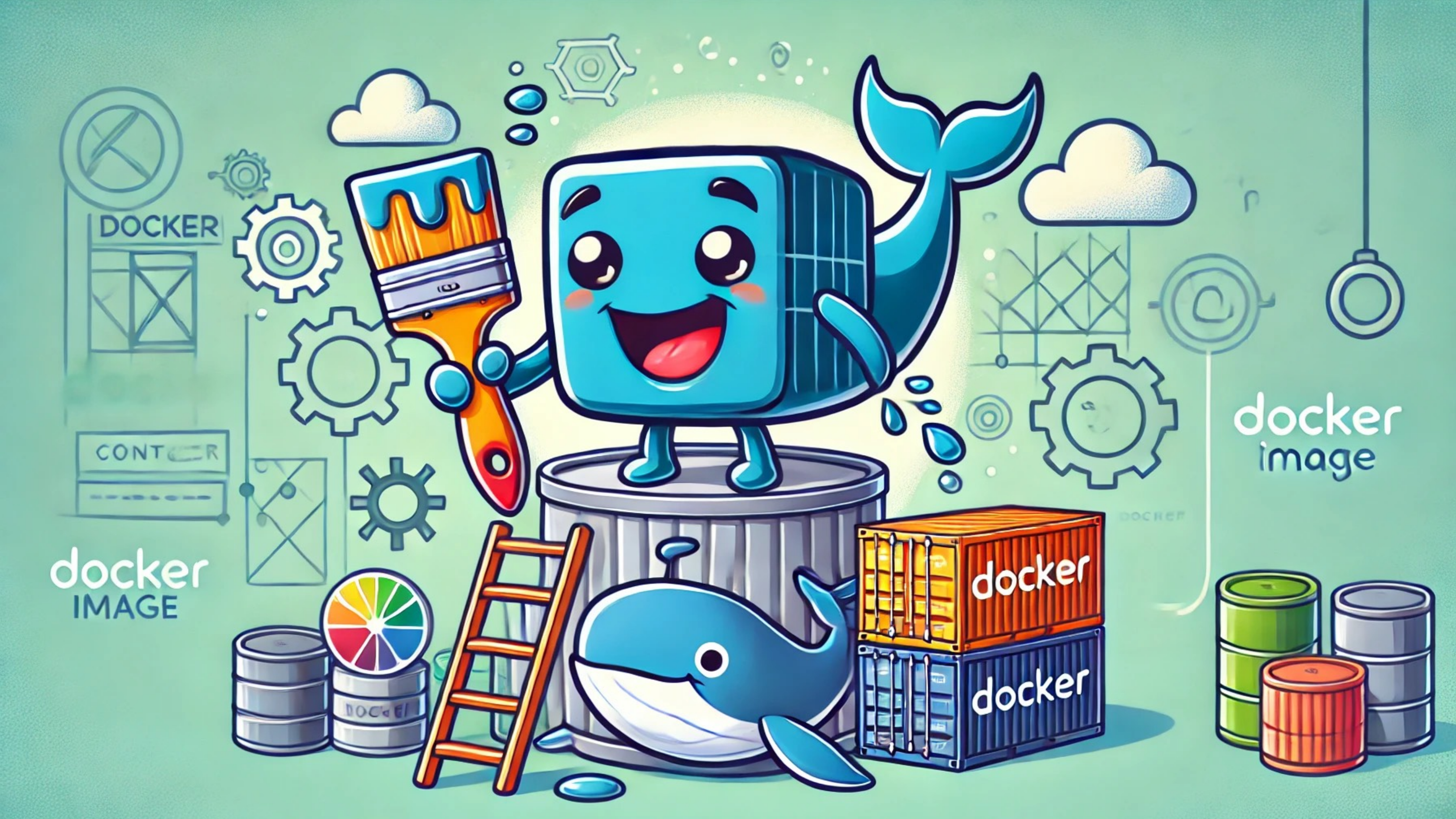
- Docker command uses the Docker APIs
 - Interact with the Daemon
- Docker is written in Go



REST-based API

- Client libraries in most languages
 - Go SDK:
 - go get github.com/docker/docker/client
 - Python
 - pip install docker
 - Unofficial: C#, Java, Ruby, Node.js, ...





DOCKER

CONTAINER

docker
IMAGE

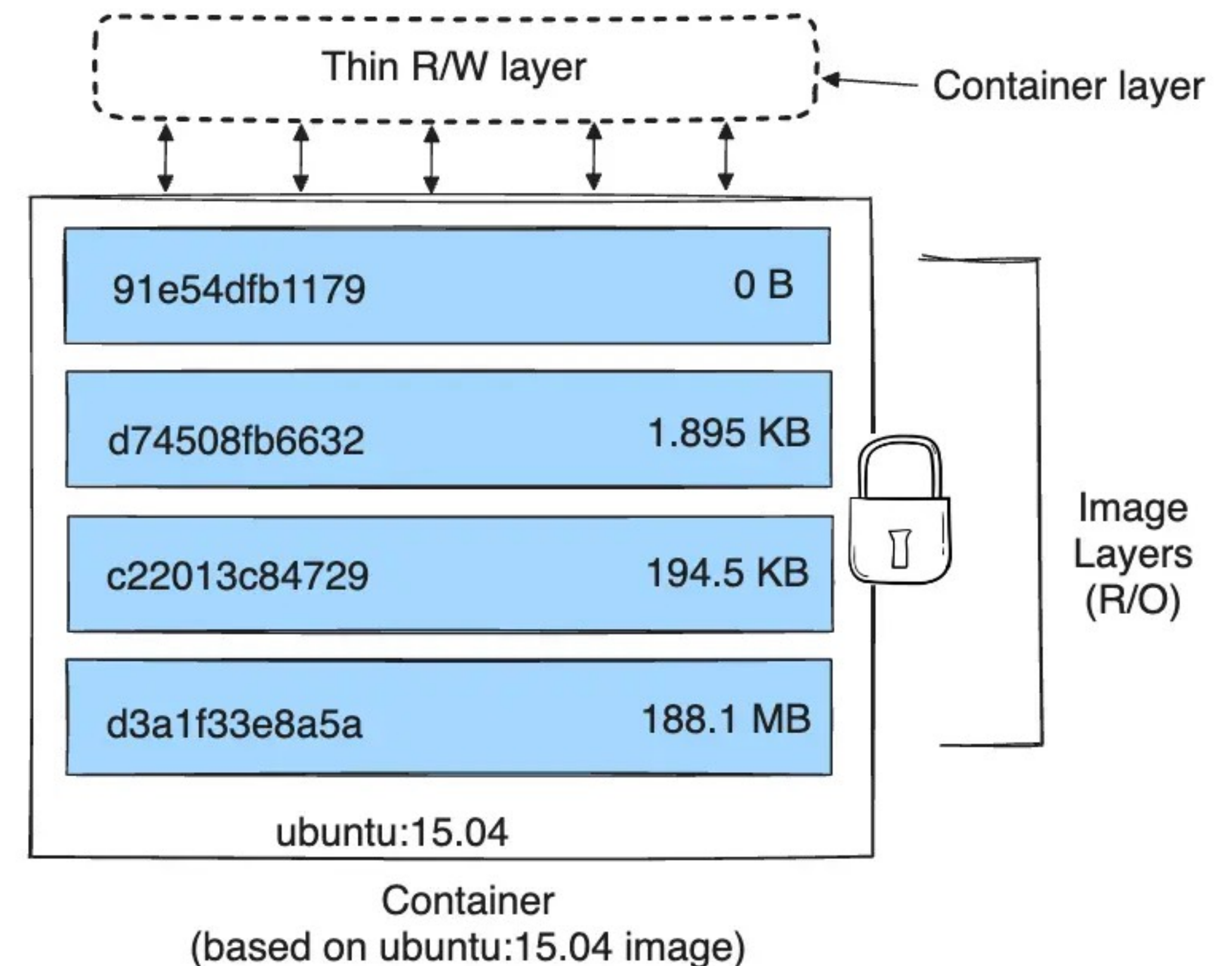
docker
image

docker

docker

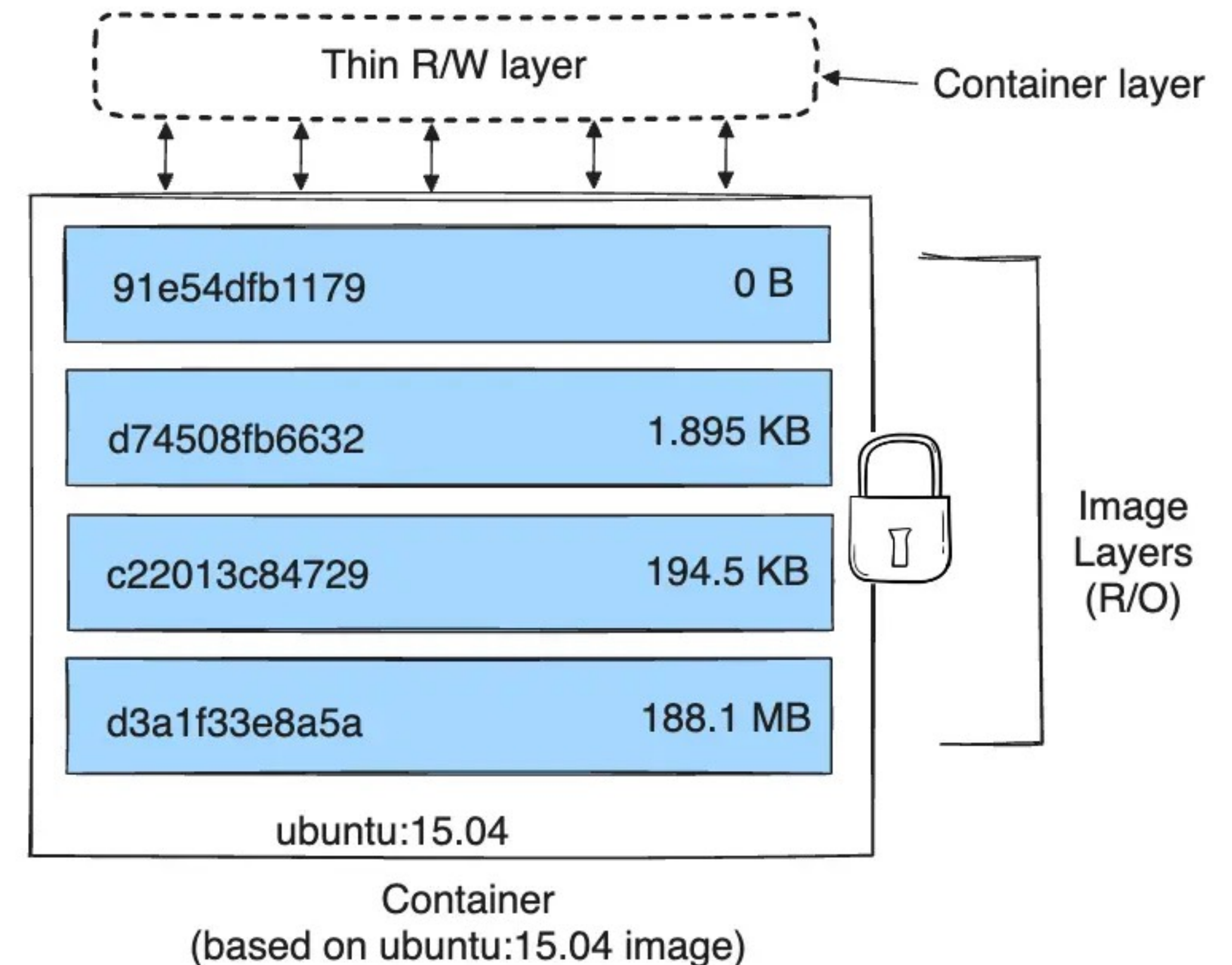
Immutable snapshot of an application and its dependencies

- Used to create containers
- Composed of layers
 - Each layer represents a change or addition to a base layer
 - Enhancing efficiency through caching



Immutable snapshot of an application and its dependencies

- Reusable: Images can be shared and reused across environments and projects
 - Images are built from scratch or pulled from repositories
- Versioned: Tagged with versions
 - Ease management and rollback



Immutable snapshot of an application and its dependencies

- Format: **[repository]:[tag]**
- Repository Name
 - Examples: nginx, golang, python, myuser/myapp
- Tag
 - Default: latest
 - Examples: nginx:1.19, golang:1.23.0-alpine3.20

Immutable snapshot of an application and its dependencies

```
docker pull golang
```

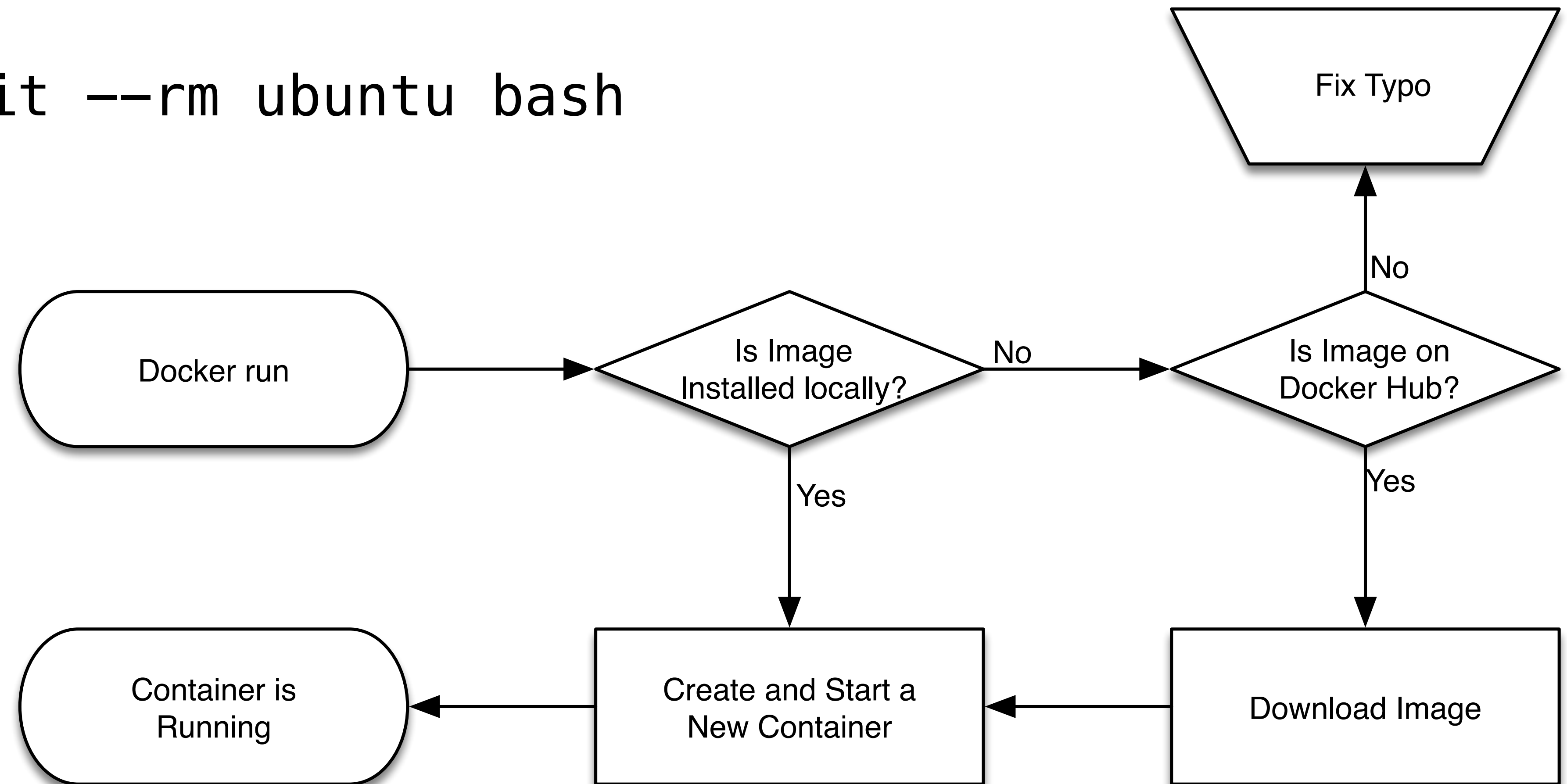
```
docker pull golang:1.25-alpine
```

```
docker pull python
```

```
docker images
```

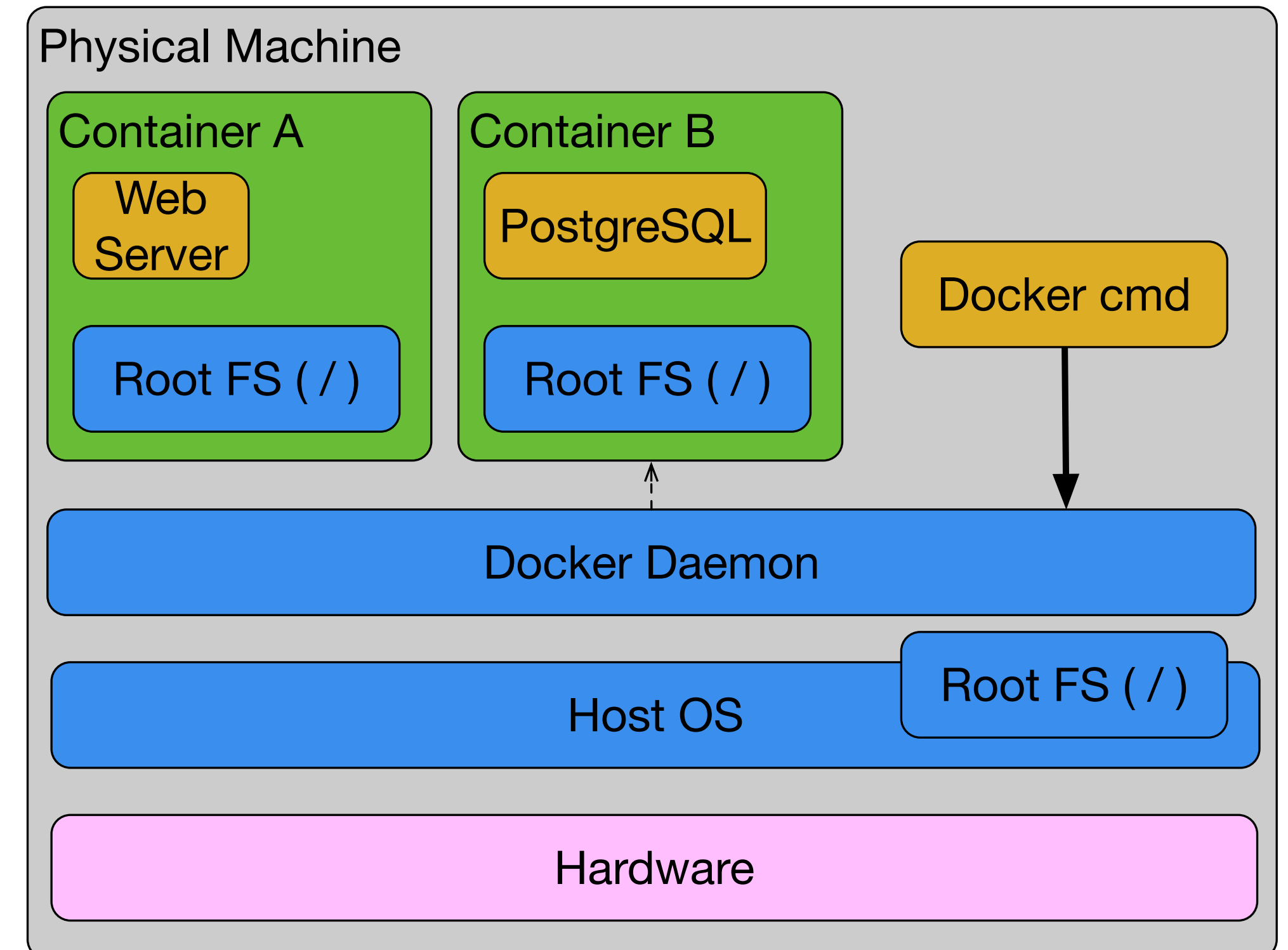
Immutable snapshot of an application and its dependencies

```
docker run -it --rm ubuntu bash
```



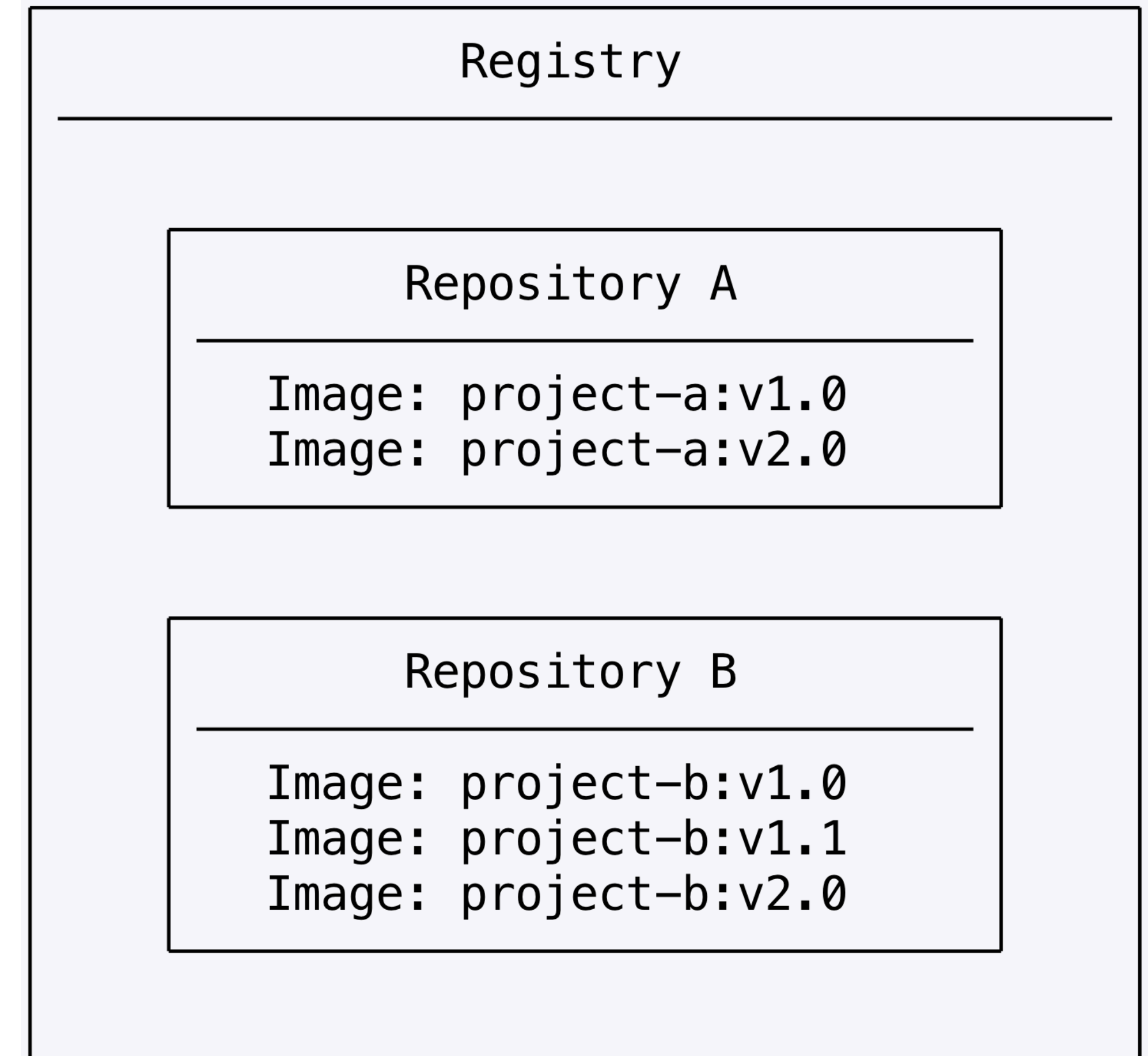
Running instance of an image

- Isolated environment for an application
- Lightweight and portable
 - Share host OS kernel
 - Minimize resource overhead
- Containers are live and ephemeral
 - Users can interact with them
 - Administrators can adjust their settings using Docker commands



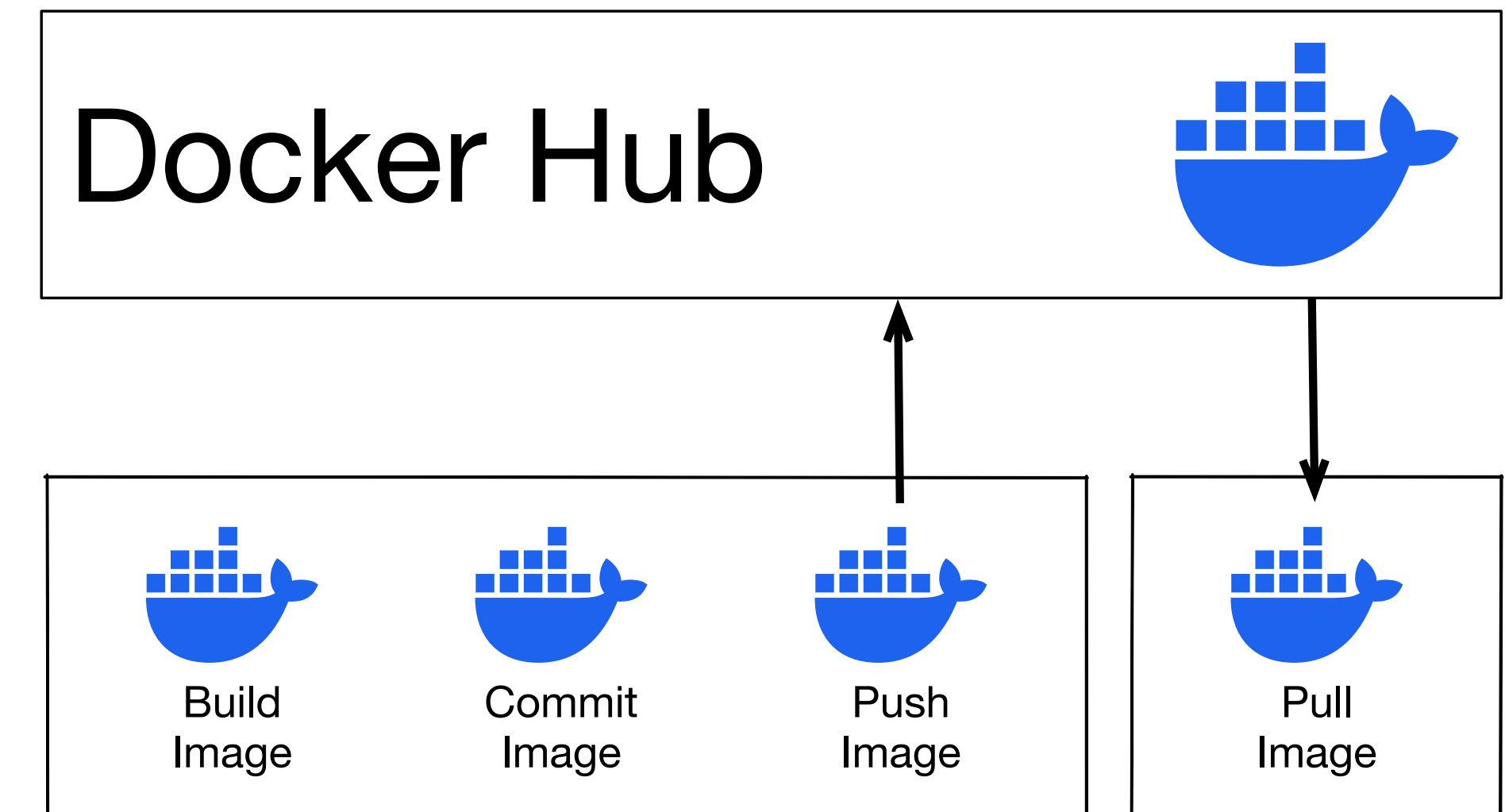
Registry, Repositories, and Images

- Registry is a centralized location that stores and manages container images
- Repository is a collection of related container images
 - Each repository contains one or more container images
 - Tagged for identification



Registry Service Provided by Docker

- Repositories for sharing container images with Docker community
- Official images
 - Clear doc, best practices, scanned for vulnerabilities...
- Publish images (external vendors)
- Private/Company images



Filters (1) [Clear All](#)

Products

- ☐ Images
- ☐ Extensions
- ☐ Plugins

Trusted Content

- ☐  Docker Official Image ⓘ
- ☐  Verified Publisher ⓘ
- ☐  Sponsored OSS ⓘ

Categories

- ☐ API Management
- ☐ Content Management System
- ☐ Data Science
- ☐ Databases & Storage
- ☐ Developer Tools
- ☐ Integration & Delivery
- ☐ Internet Of Things
- ☐ Languages & Frameworks
- ☐ Machine Learning & AI
- ☒ Message Queues
- ☐ Monitoring & Observability

1 - 25 of 668 available results.

Message Queues ×

Suggested ⌵

	rabbitmq  Updated 2 days ago RabbitMQ is an open source multi-protocol messaging broker. MESSAGE QUEUES	↓1B+ · ☆5.1K	Pulls: 2,976,059 Last week  Learn more ↗
	nats  Updated 3 days ago NATS is an open-source, high-performance, cloud native messaging system. MESSAGE QUEUES	↓100M+ · ☆493	Pulls: 216,703 Last week  Learn more ↗
	eclipse-mosquitto  Updated 24 days ago Eclipse Mosquitto is an open source message broker which implements MQTT version 5, 3.1.1 and 3.1 MESSAGE QUEUES	↓500M+ · ☆1.3K	Pulls: 410,966 Last week  Learn more ↗
	emqx  Updated 3 days ago The most scalable open-source MQTT broker for IoT, IIoT, connected vehicles, and more. MESSAGE QUEUES	↓1M+ · ☆87	Pulls: 25,970 Last week  Learn more ↗

Joke time

Why did the Docker container
break up with the virtual machine?

It needed some space and
felt they were too heavy



SNL

Discuss with your neighbor!

Discussion Topics

1. Why do you think developers might want a tool like Docker Desktop on their laptops?
2. Have you ever had trouble getting software to run on your computer? How might containers help with that?
3. If you could “package” an app you use every day to share with a friend, which one would it be?
4. What do you think might be the difference between installing software directly vs. running it inside Docker?
5. Have you tried Docker Desktop?

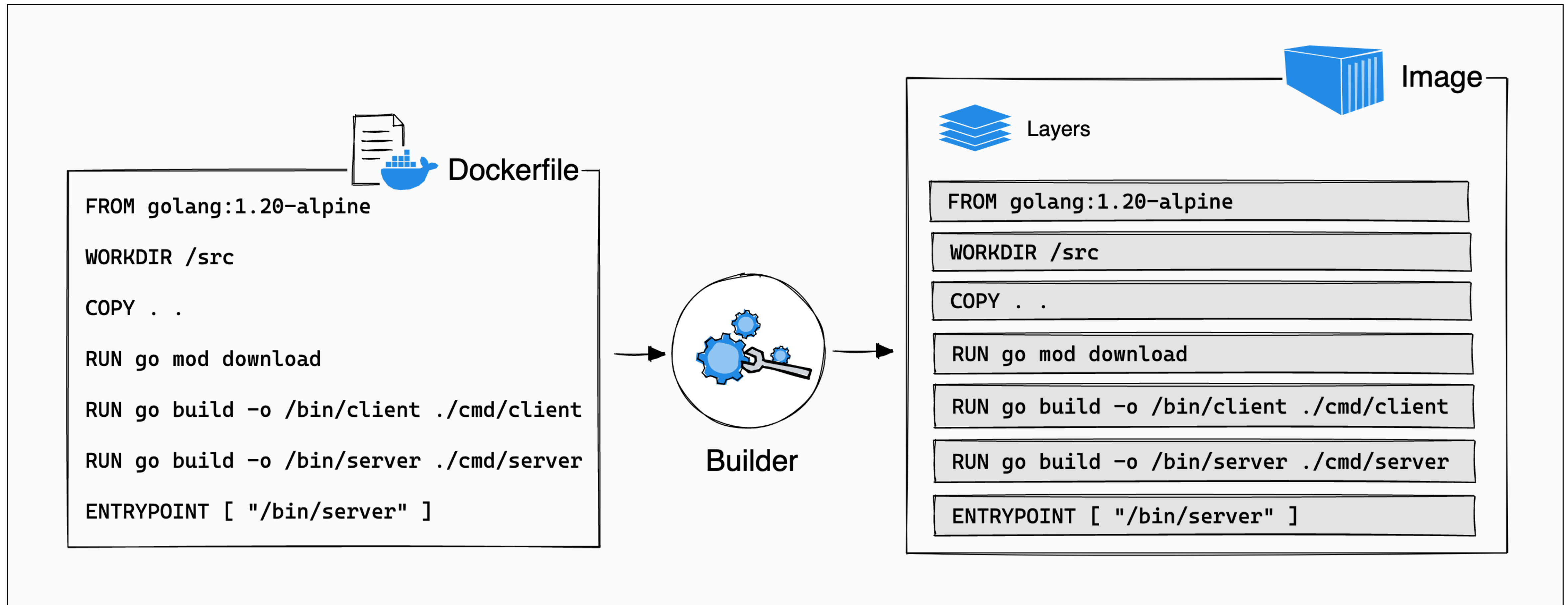
The Dockerfile

Text File for Image Construction

- Docker builds images based on Dockerfile
- Standard set of commands

Instruction	Description
<code>ADD</code>	Add local or remote files and directories.
<code>ARG</code>	Use build-time variables.
<code>CMD</code>	Specify default commands.
<code>COPY</code>	Copy files and directories.
<code>ENTRYPOINT</code>	Specify default executable.
<code>ENV</code>	Set environment variables.
<code>EXPOSE</code>	Describe which ports your application is listening on.
<code>FROM</code>	Create a new build stage from a base image.
<code>HEALTHCHECK</code>	Check a container's health on startup.
<code>LABEL</code>	Add metadata to an image.
<code>MAINTAINER</code>	Specify the author of an image.
<code>ONBUILD</code>	Specify instructions for when the image is used in a build.
<code>RUN</code>	Execute build commands.
<code>SHELL</code>	Set the default shell of an image.
<code>STOPSIGNAL</code>	Specify the system call signal for exiting a container.

Text File for Image Construction



Caching, Efficient Storage, Incremental Changes

- Layers represent changes in a Docker image
 - Each command in a Dockerfile (like RUN, COPY, ADD) creates a new layer
- Layers are shared across images (for efficiency)
 - Layers are stored only once
 - Faster image builds
- Layer caching speeds up image builds
 - Unmodified layers are reused

Cached Layers



Layers

Cache?

```
FROM golang:1.20-alpine
```



```
WORKDIR /src
```



```
COPY . .
```



```
RUN go mod download
```



```
RUN go build -o /bin/client ./cmd/client
```



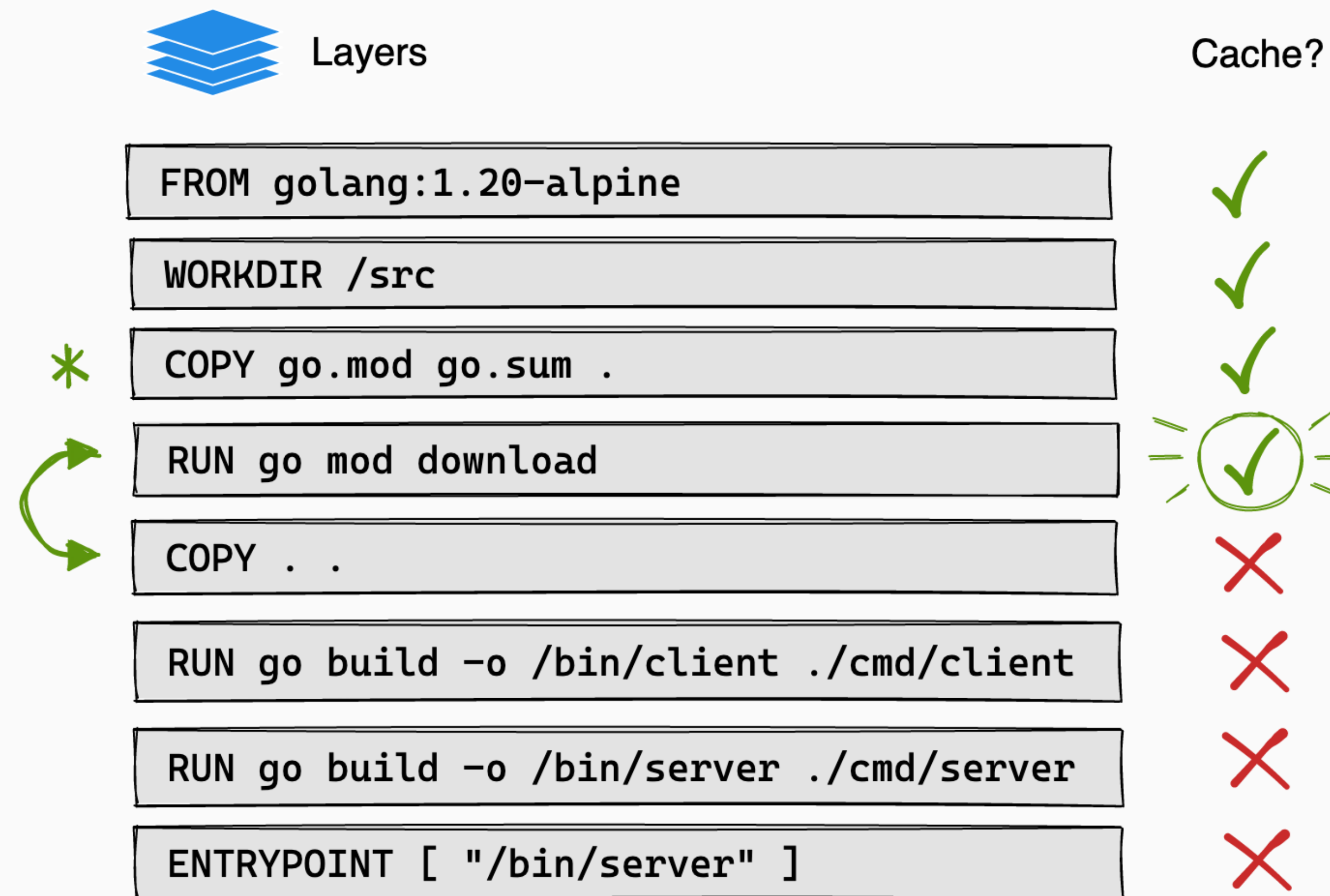
```
RUN go build -o /bin/server ./cmd/server
```



```
ENTRYPOINT [ "/bin/server" ]
```



Instruction order matters



Docker Networking

The Network Subcommand

```
docker network ls
```

```
docker network inspect bridge
```

```
docker network inspect host
```

Default networks

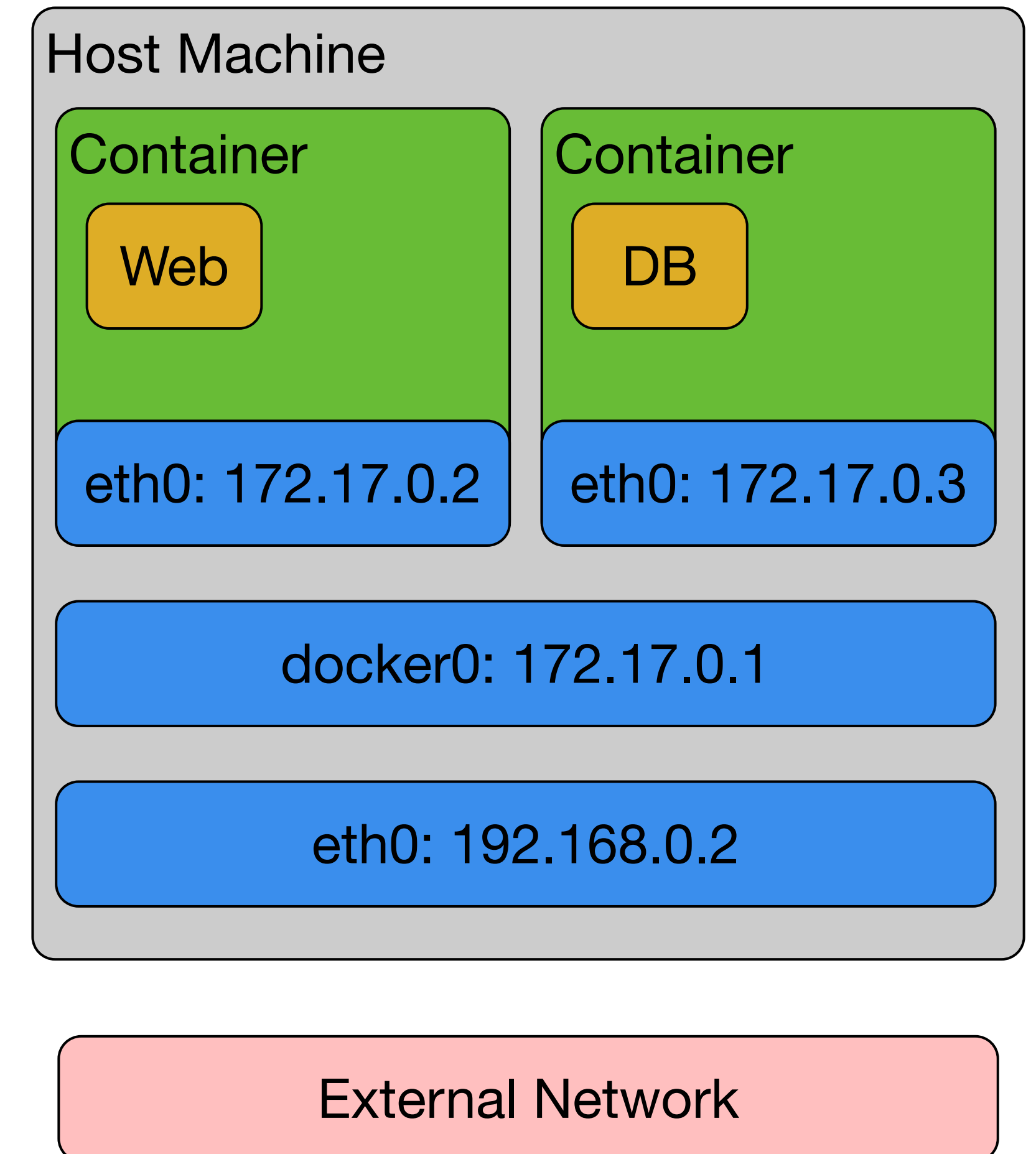
```
docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
9dcd3906d274	bridge	bridge	local
22096b6a3066	host	host	local
f10efee3f300	none	null	local

Bridge Networking (Default)

- Network for containers on a single host (few hosts)
- Provides **isolated network** among containers
 - Automatic IP addressing
- External access via **port mapping**
- Slight overhead compared to host networking

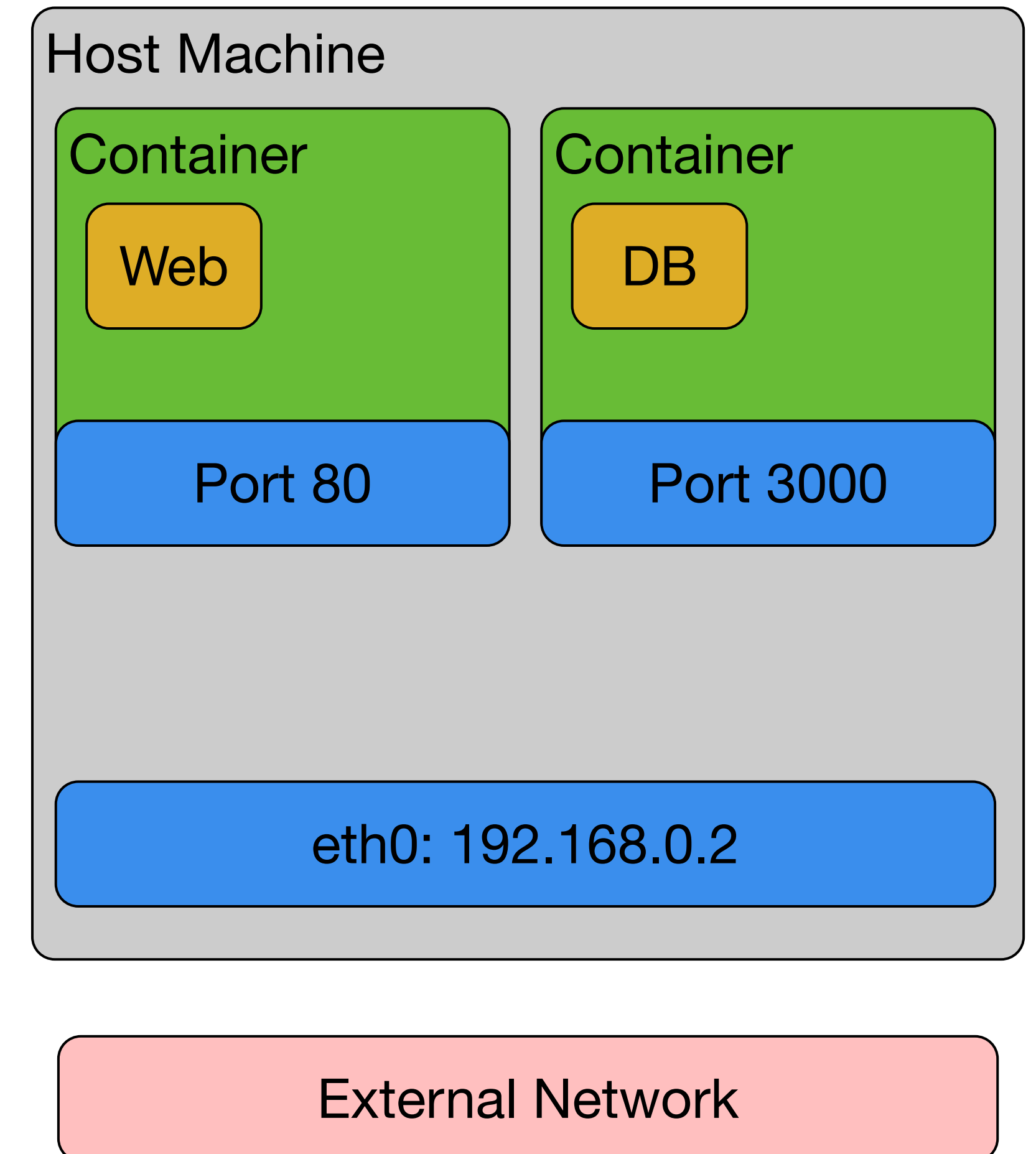
```
docker run ubuntu
```



Host Networking

- Containers **share** the host's **network stack**
- Reduces network isolation
 - Share host's network namespace (IP, port)
 - Potentially **less secure**
- **Better performance** than bridge networking

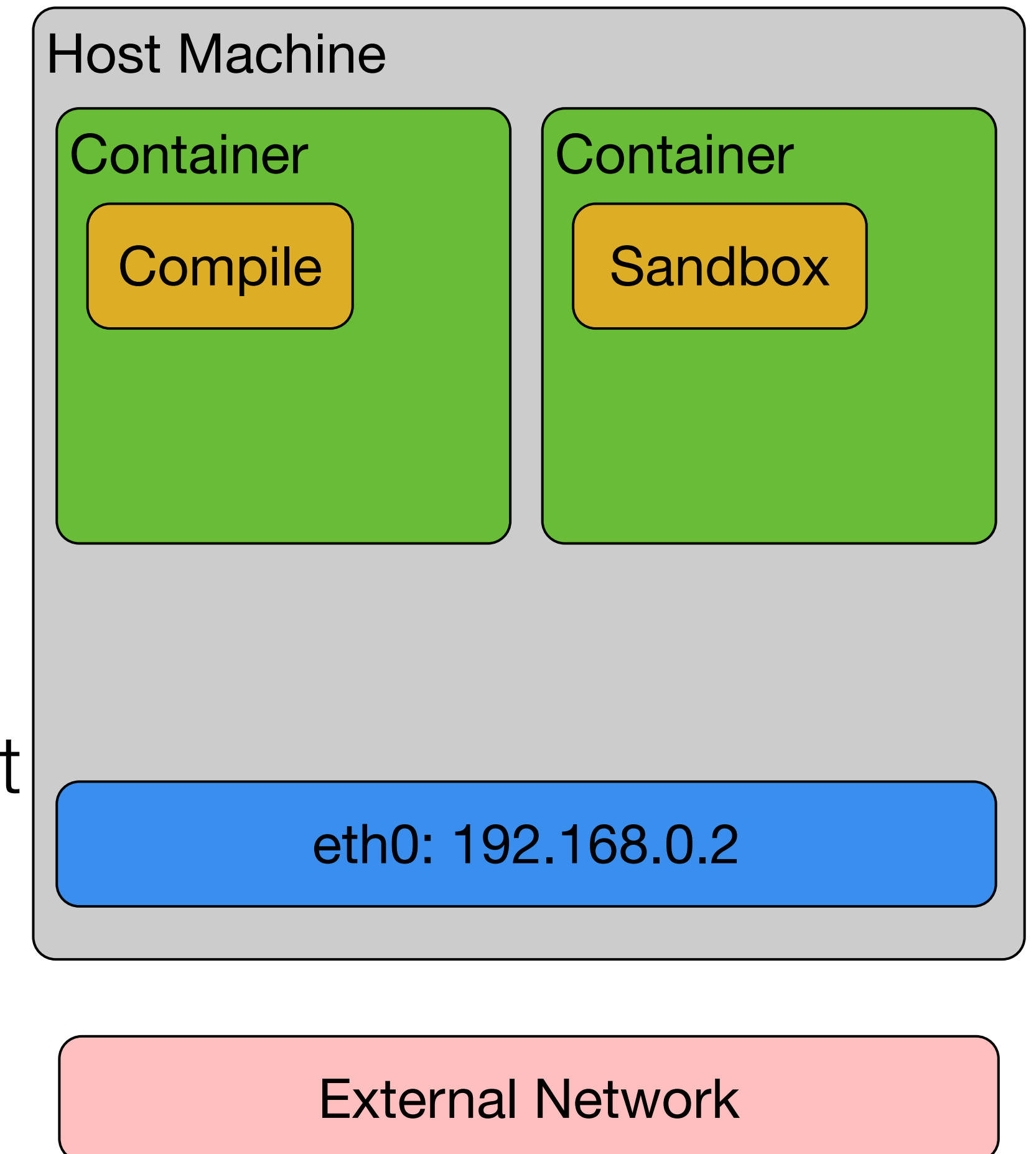
```
docker run --network=host ubuntu
```



None Network

- Containers with **complete network isolation**
- Useful for tasks that don't require network access
 - Consistent and reproducible builds
 - Difficult to configure dependencies and environment
- **Output to file system** only

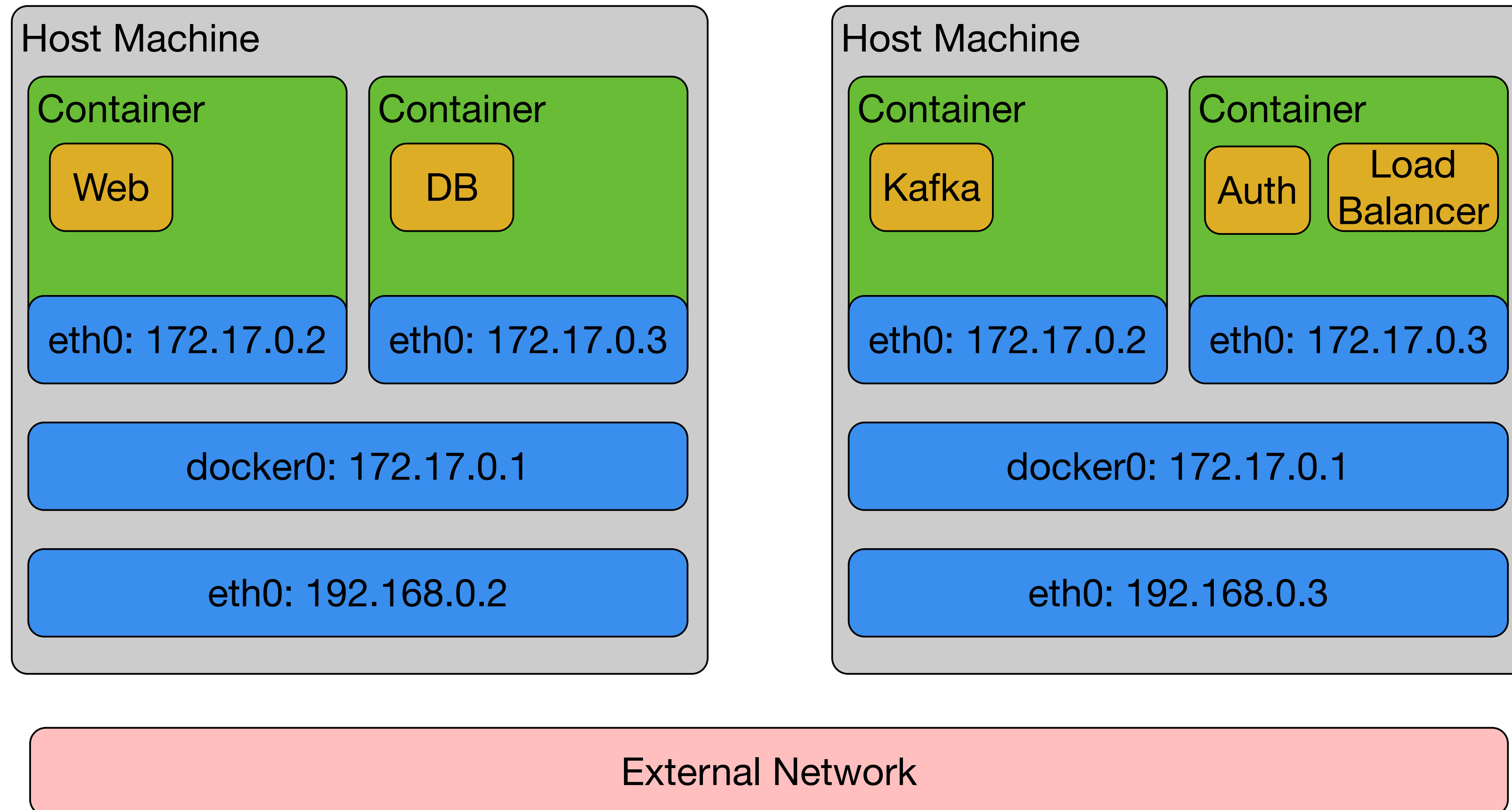
```
docker run --network=none ubuntu
```



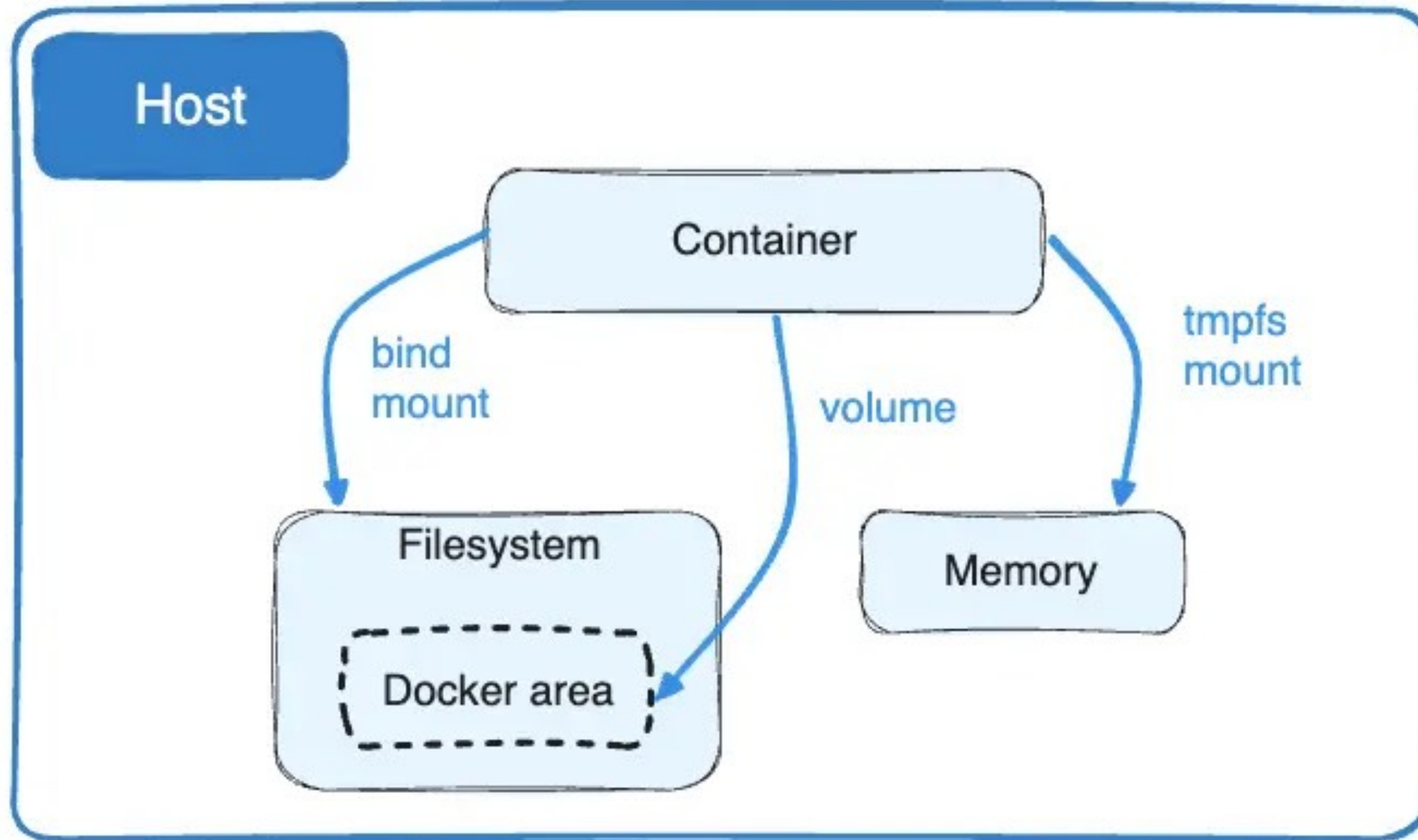
Overlay Networking

- Multi-host networking
 - Communication between **containers on different hosts**
- Automatic service discovery and configuration
- Hosts managed by Docker Swarm or Kubernetes clusters

Overlay Networking

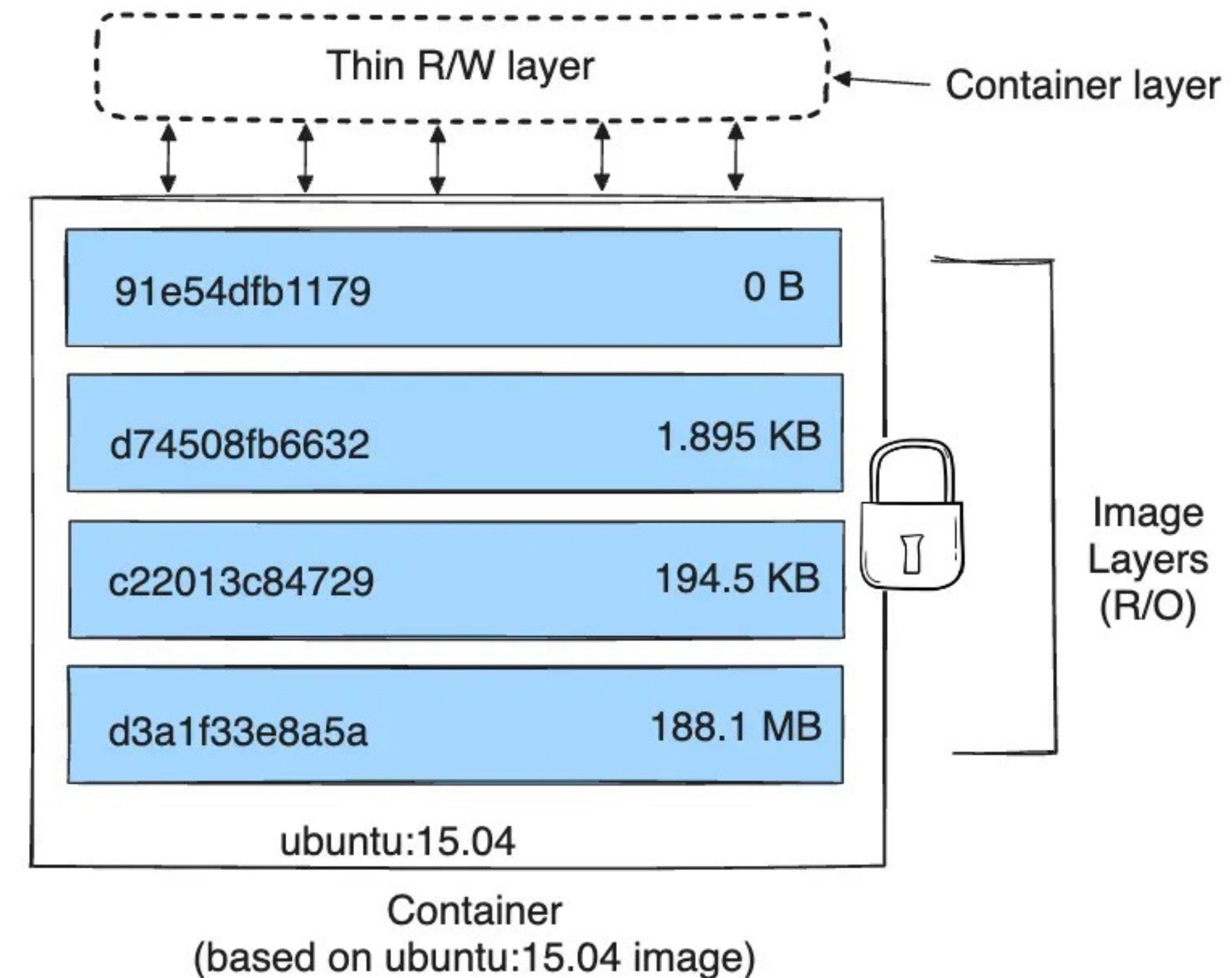


Docker Storage



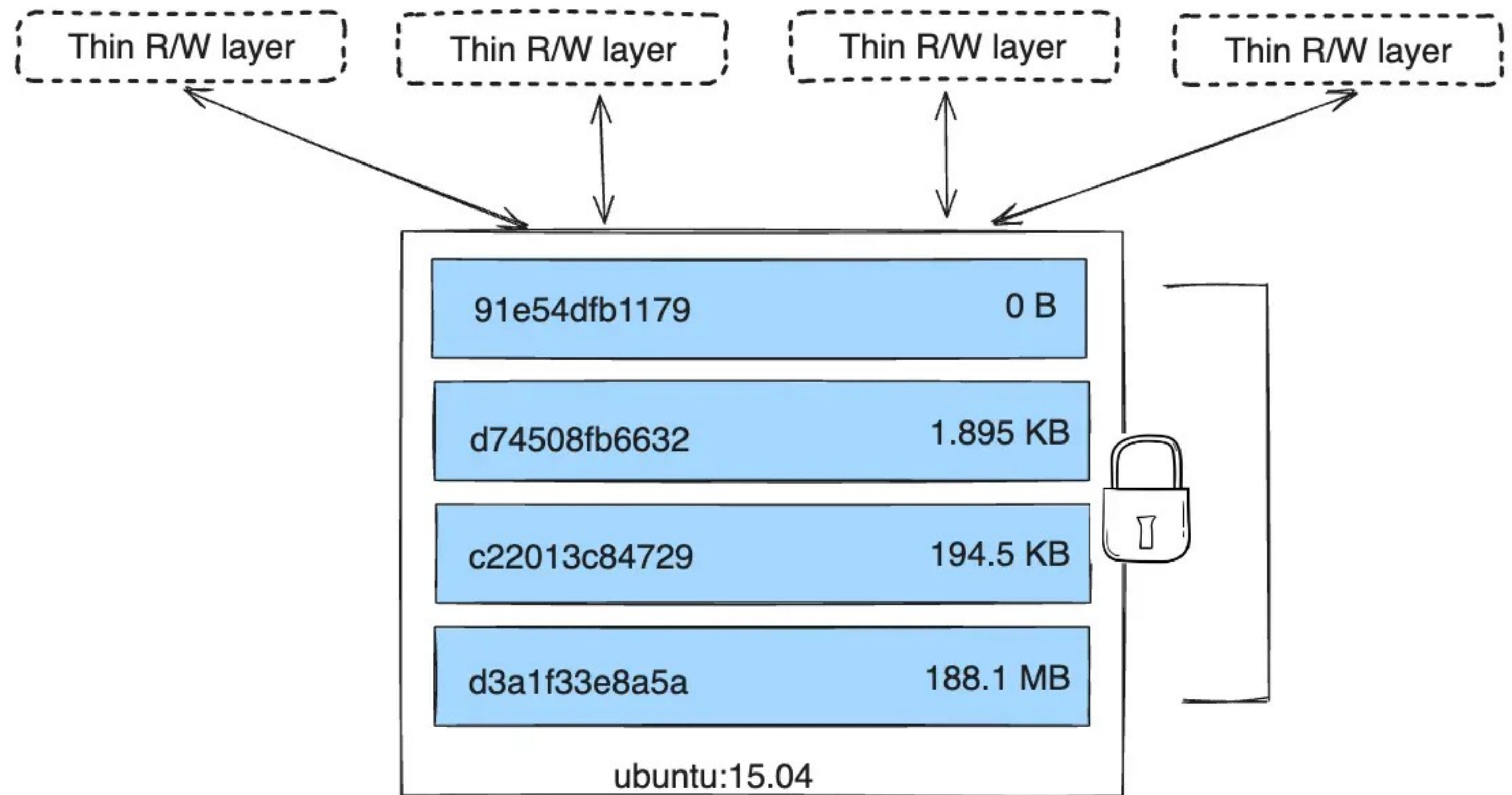
Layered File System

- Ephemeral storage
- **Data retained** across stops and **restarts** of the container
- **Data is lost** when the container is **deleted**
 - Mainly useful for temporary data



Layered File System

- Different containers can use ubuntu:15.04
- Each container get their own writable layer



Data stored in the host's memory (RAM)

- Ephemeral storage
- **Data is lost** when the container is **stopped**
 - Useful for caches or session data

Persistent data storage outside container's filesystem

- **Persistent Storage**

- Ensure data persists across container restarts
- Managed by Docker `/var/lib/docker/volumes`

- **Shared Access**

- Share data between containers by mounting the same volume
- Share data with host system
- Decouple data from container lifecycle, easy backups, and migration

- **Map specific file/directory** on **host's filesystem** to a path inside the container
- Use Cases: Access specific content on host file system
 - Source code and configuration files
- Tightly coupled to host's filesystem structure
 - Less portable than Docker managed volumes

tree -L1 courses

```

courses
├── dat240-2023
├── dat240-2024
├── dat310-2023
├── dat310-2024
├── dat320-2023
├── dat320-2024
├── dat515-2024
├── dat520-2023
├── dat520-2024
├── dat600-2024
└── qf-103
  
```

courses/dat515-2024/

```

├── assignments
│   ├── lab1
│   ├── lab2
│   ├── lab3
│   ├── lab4
│   ├── lab5
│   └── lab6
└── tests
    ├── lab1
    ├── lab2
    ├── lab3
    ├── lab4
    ├── lab5
    └── lab6
  
```

courses/dat320-2023/

```

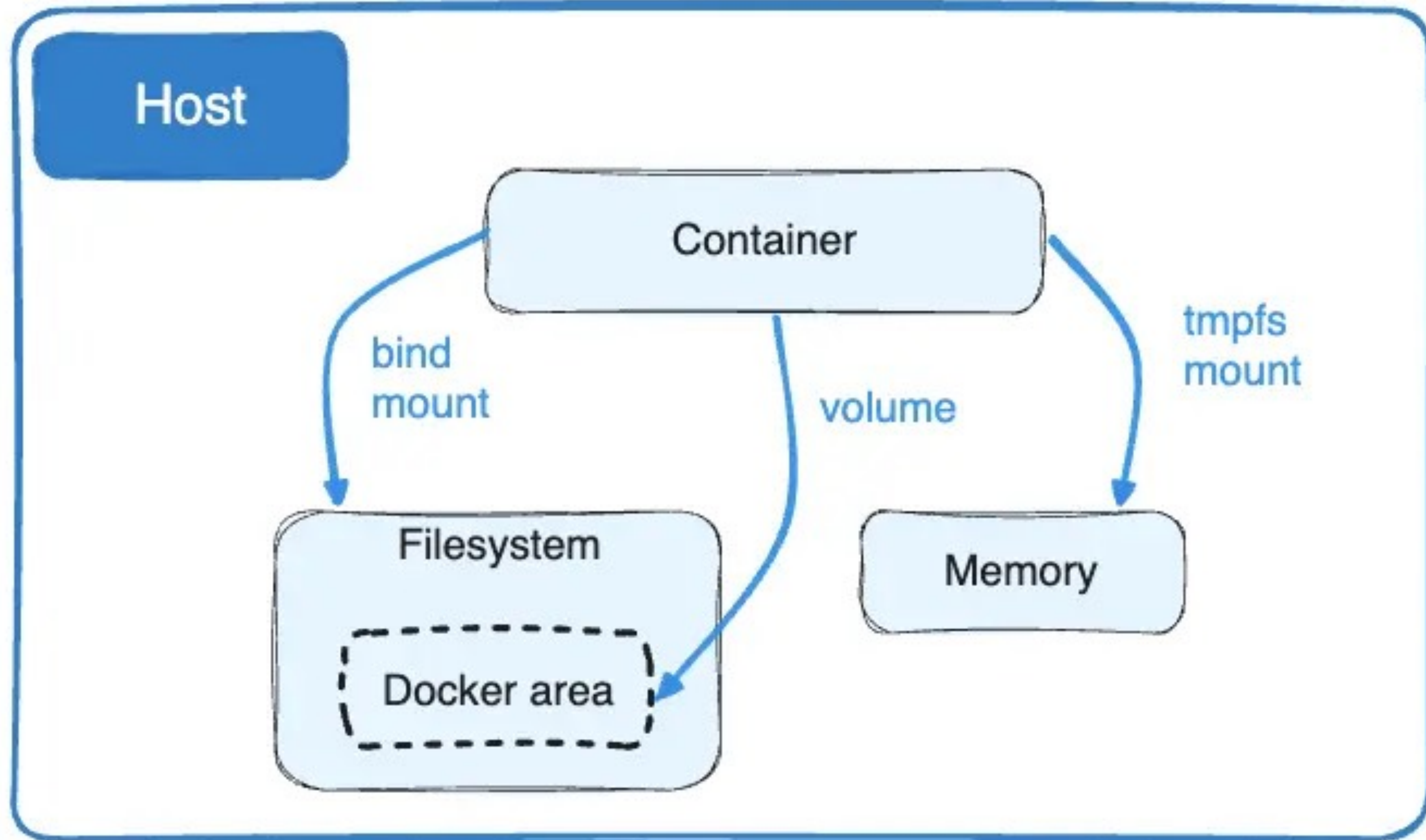
├── assignments
│   ├── go.mod
│   ├── go.sum
│   ├── lab1
│   │   ├── generate_token
│   │   ├── git_questions.md
│   │   ├── missing_semester_questions.md
│   │   ├── README.md
│   │   └── shell_questions.md
│   ├── lab2
│   │   ├── c_questions.md
│   │   ├── hello
│   │   └── README.md
│   └── lab3
│       ├── cipher
│       ├── cmd
│       ├── errors
│       ├── go_questions.md
│       ├── multiwriter
│       ├── README.md
│       ├── sequence
│       └── stringer
└── tests
  
```

└── tests

```

├── go.mod
├── go.sum
├── internal
│   ├── git
│   ├── staff
│   └── token
├── lab1
│   ├── assignment.yml
│   ├── git_questions_ag_test.go
│   ├── main_ag_test.go
│   ├── missing_semester_ag_test.go
│   ├── shell_questions_ag_test.go
│   ├── token_ag_test.go
│   └── tokengen
├── lab2
│   ├── assignment.yml
│   ├── c_questions_ag_test.go
│   ├── hello_world_ag_test.go
│   ├── hello_world_init.go
│   └── main_ag_test.go
└── scripts
    ├── Dockerfile
    └── run.sh
  
```

Recap: Docker Storage Options



Summary of Storage Alternatives

Storage Option	Description	Usage	Persistence Across Start/Stop	Persistence After Container Removal
Volumes	Managed by Docker, stored outside the container's filesystem	Sharing data between containers, backups, migration, decoupling from lifecycle	Yes	Yes
Bind Mounts	Directly maps a host directory or file to a path inside a container	Accessing specific host files or directories, development (e.g., syncing code)	Yes	Yes
tmpfs Mounts	Stores data in the host's memory (RAM), not on disk	Temporary data that doesn't need persistence (e.g., caching, session data)	No	No
Layered Filesystem	Manages container's filesystem layers; data in the writable layer	Default container storage, ephemeral data storage within the container	Yes	No

Working with Docker's Go SDK

Docker Daemon must be
running for the following
examples

<https://pkg.go.dev/github.com/docker/docker>

Implement your own CI tool with official Docker SDK

```
$ mkdir docker-ci; cd docker-ci
```

```
$ go mod init github.com/meling/docker-ci
```

```
$ go get github.com/docker/docker
```

```
$ cat go.mod
```

```
func TestPing(t *testing.T) {
    c, err := client.NewClientWithOpts(
        client.FromEnv,
        client.WithAPIVersionNegotiation(),
    )
    if err != nil {
        t.Fatalf("Failed to create container client: %v", err)
    }
    if _, err := c.Ping(t.Context()); err != nil {
        t.Fatalf("Failed to ping Docker daemon: %v", err)
    }
    t.Log("Ping successful")
}
```

<https://pkg.go.dev/github.com/docker/docker>

Implement your own CI tool with official Docker SDK

```
$ mkdir docker-ci; cd docker-ci
```

```
$ go mod init github.com/meling/docker-ci
```

```
$ go get github.com/docker/docker
```

```
$ cat go.mod
```

```
$ go mod tidy
```

```
$ go test -v
```

```
$ cat go.mod
```

<https://pkg.go.dev/github.com/relab/container>

```
# Implement your own CI tool
$ mkdir ci; cd ci
$ go mod init github.com/meling/ci
$ go get github.com/relab/container
$ cat go.mod
```

Hein's Stripped-down Docker Go SDK

```
package ci

import (
    "testing"

    "github.com/relab/container"
)

run test | debug test
func TestPing(t *testing.T) {
    c, err := container.NewContainer()
    if err != nil {
        t.Fatalf("Failed to create container client: %v", err)
    }

    if err := c.Ping(t.Context()); err != nil {
        t.Fatalf("Failed to ping Docker daemon: %v", err)
    }
    t.Log("Ping successful")
}
```

```
func TestPing(t *testing.T) {
    c, err := container.NewContainer()
    if err != nil {
        t.Fatalf("Failed to create container client: %v", err)
    }

    if err := c.Ping(t.Context()); err != nil {
        t.Fatalf("Failed to ping Docker daemon: %v", err)
    }
    t.Log("Ping successful")
}
```

```
func TestPing(t *testing.T) {
    c, err := client.NewClientWithOpts(
        client.FromEnv,
        client.WithAPIVersionNegotiation(),
    )
    if err != nil {
        t.Fatalf("Failed to create container client: %v", err)
    }
    if _, err := c.Ping(t.Context()); err != nil {
        t.Fatalf("Failed to ping Docker daemon: %v", err)
    }
    t.Log("Ping successful")
}
```

Joke time

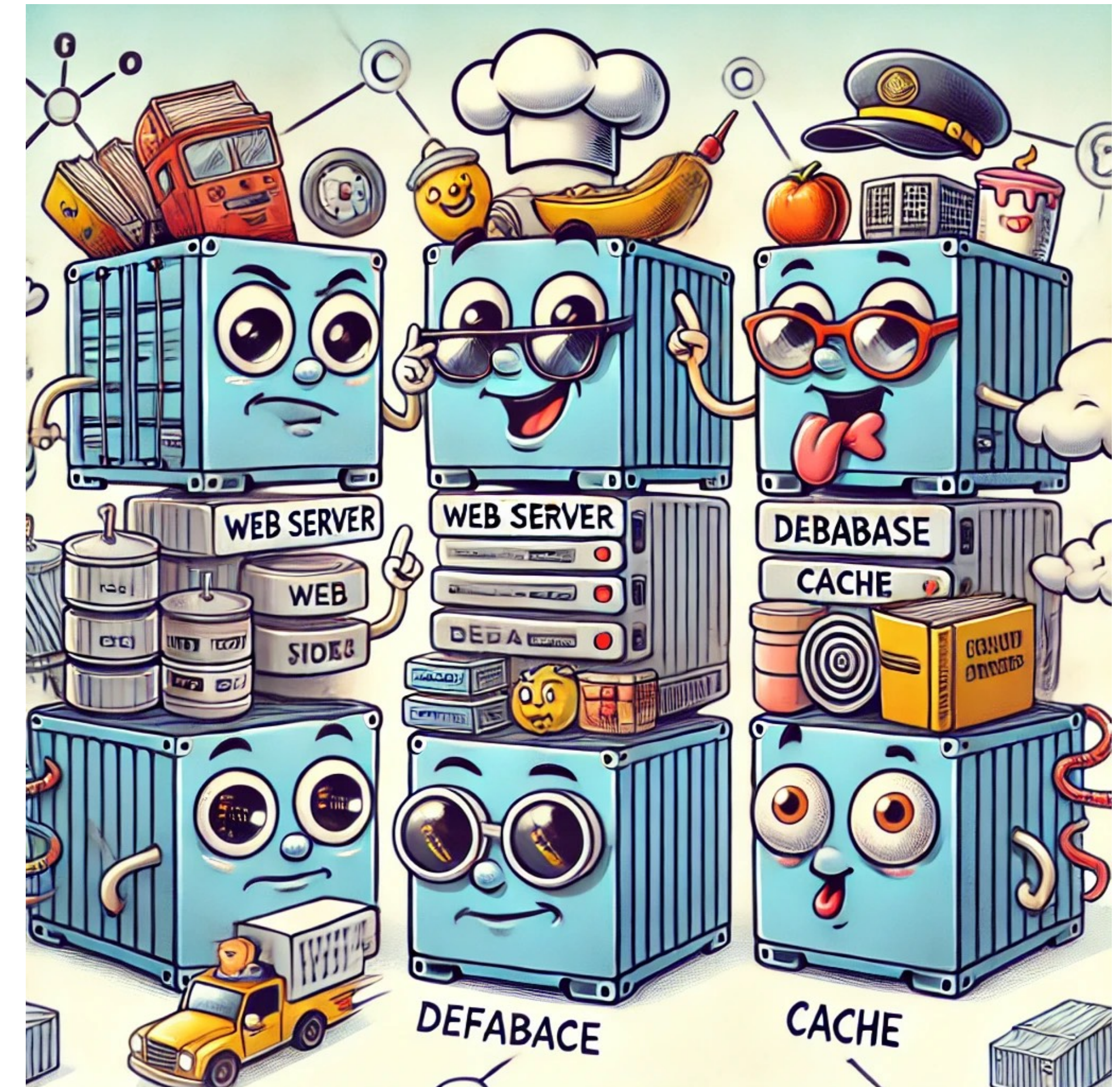
Why do Docker containers
never get lonely?

Because they always have
a lot of “friends”
(processes) running inside.

Multi-Container Applications

Microservice Architecture

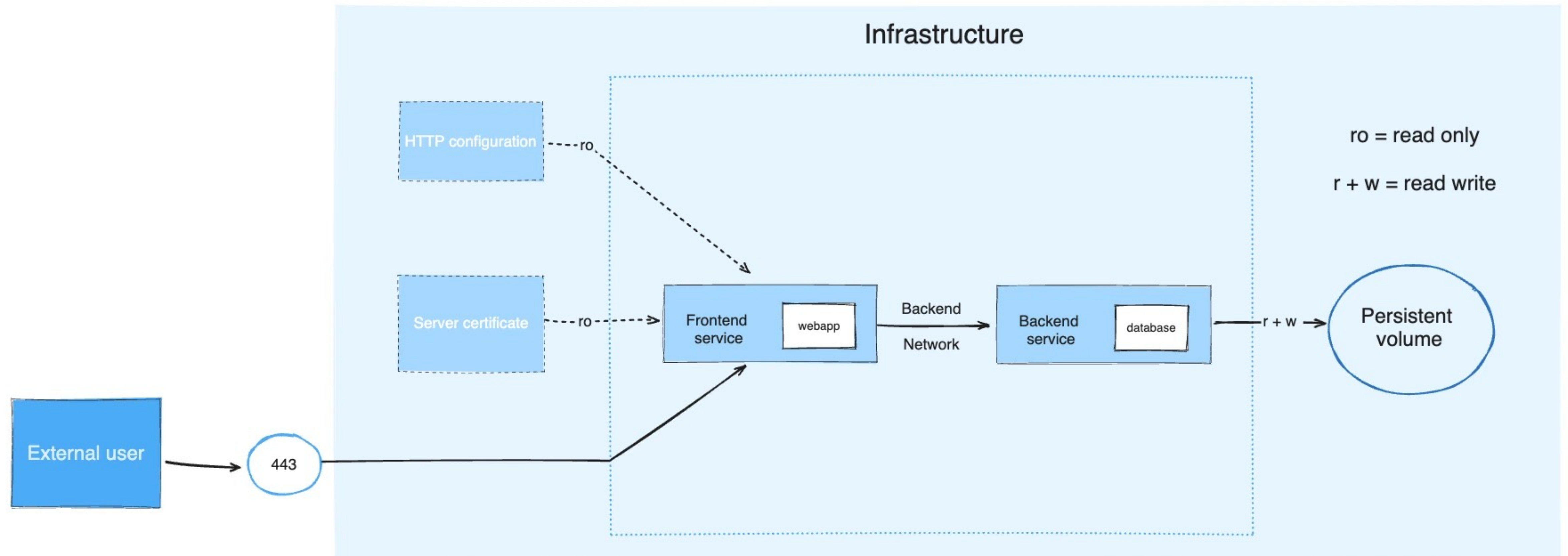
- **Single Responsibility per Container**
 - Run only one service to ensure modularity and easier management
- **Inter-Container Communication**
 - Containers communicate with each other over a Docker network
 - Internal DNS to resolve service names to container IPs, making services discoverable



Microservice Architecture

- **Loose Coupling**
 - Isolating services into different containers
 - Updates can be made to one service without affecting other containers
 - More maintainable and resilient applications
- **Scaling and Resilience**
 - Individual services can be scaled independently based on load
 - Load balancer, multiple web servers, a single database

- Docker Compose is a tool that simplifies managing multi-container applications
- A `compose.yml` file to define and run multiple containers
 - As a single service stack,
 - handling container creation,
 - networking, and
 - volume management



```
services:
  frontend:
    image: example/webapp
    ports:
      - "443:8043"
    networks:
      - front-tier
      - back-tier
    configs:
      - httpd-config
    secrets:
      - server-certificate

  backend:
    image: example/database
    volumes:
      - db-data:/etc/data
    networks:
      - back-tier
```

```
volumes:
  db-data: {}

configs:
  httpd-config:
    external: true

secrets:
  server-certificate:
    external: true

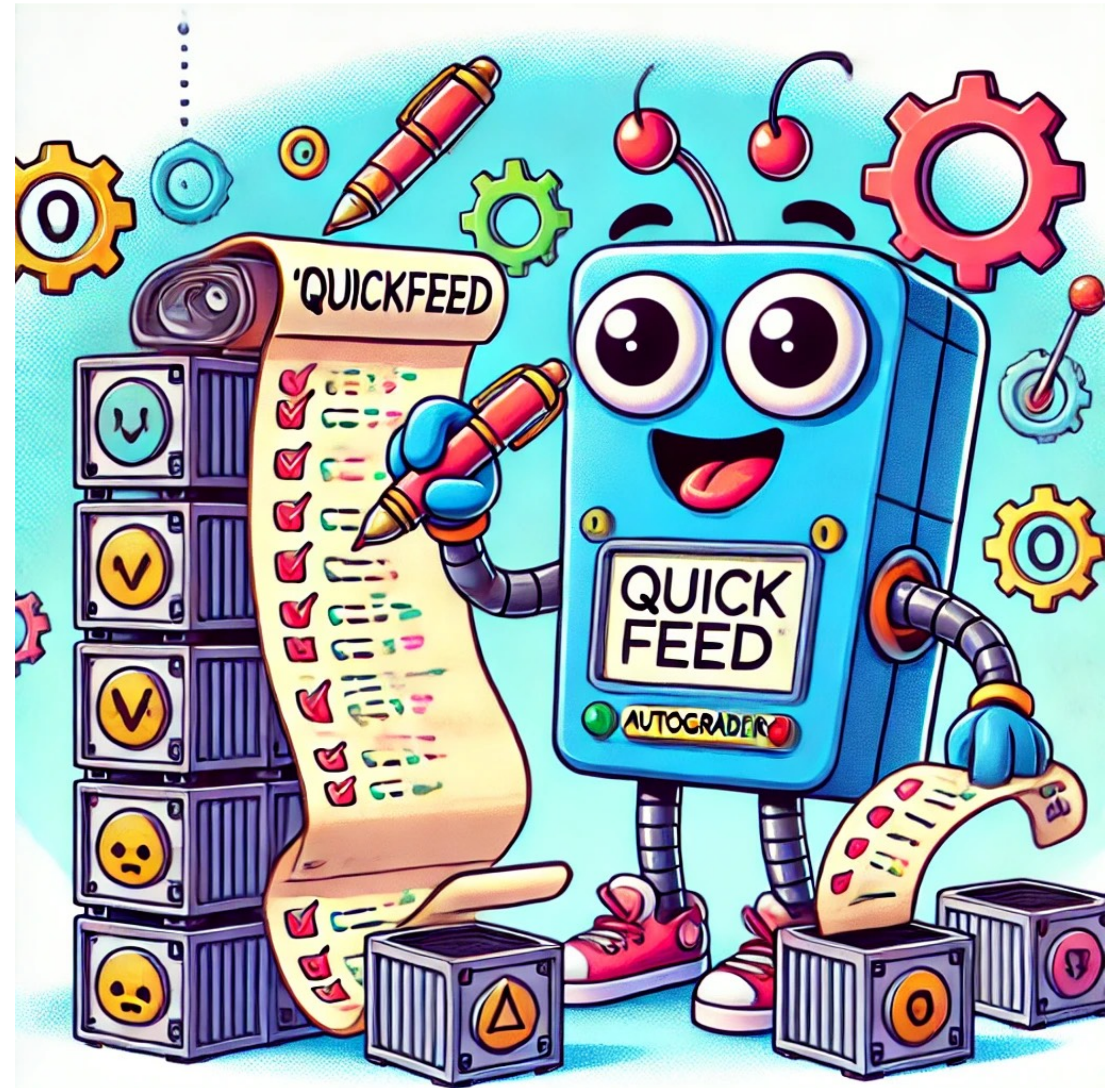
networks:
  front-tier: {}
  back-tier: {}
```

Separation of Secret Management from Container Spec

- Avoid hardcoding into the compose file
- Make the application more secure and flexible
- Secret resources managed externally via Docker Swarm

```
docker swarm init
docker secret create server-certificate /path/to/server-certificate.pem
docker secret ls
docker-compose up -d
```

QuickFeed Interlude



```
# syntax=docker/dockerfile:1.4
FROM golang:alpine
```

```
RUN apk update && apk add --no-cache git bash build-base docker openrc
```

```
WORKDIR /quickfeed
```

The run.sh script

```
#image/dat515

# The script may use the following environment variables:
#
# TESTS          - to access the tests (cloned from the course's tests repository)
# ASSIGNMENTS    - to access the assignments (cloned from the course's assignments repository)
# SUBMITTED      - to access the student's or group's submitted code (cloned from their repository)
# CURRENT        - name of the current assignment folder
#
# Note that the above folders are copied into the container for each test run.
# Thus, the script is free to modify them as needed.

start=$SECONDS
printf "*** Preparing Test Execution for %s ***\n" "$CURRENT"

# Move to folder with submitted code for the current assignment to test.
cd "$SUBMITTED/$CURRENT"
# Remove student written tests to avoid interference
find . -name '*_test.go' -exec rm -rf {} \;

# Copy tests into student assignments folder for running tests
cp -r "$TESTS"/* "$SUBMITTED"/

# $TESTS does not contain go.mod and go.sum: make sure to get the kit/score package
go get -t github.com/quickfeed/quickfeed/kit/score
go mod tidy

printf "\n*** Finished Test Setup in %s seconds ***\n" "$(( SECONDS - start ))"
start=$SECONDS
printf "\n*** Running Tests ***\n\n"
go test -v -timeout 240s ./... 2>&1
printf "\n*** Finished Running Tests in %s seconds ***\n" "$(( SECONDS - start ))"
```

```
// NewDockerCI returns a runner to run CI tests.
func NewDockerCI(logger *zap.SugaredLogger) (*Docker, error) {
    cli, err := client.NewClientWithOpts(
        client.FromEnv,
        client.WithAPIVersionNegotiation(),
    )
    if err != nil {
        return nil, err
    }
    return &Docker{
        client: cli,
        logger: logger,
    }, nil
}
```

```
// Run implements the CI interface. This method blocks until the job has been
// completed or an error occurs, e.g., the context times out.
func (d *Docker) Run(ctx context.Context, job *Job) (string, error) {
    if d.client == nil {
        return "", fmt.Errorf("cannot run job: %s; docker client not initialized", job.Name)
    }

    resp, err := d.createImage(ctx, job)
    if err != nil {
        return "", err
    }
    d.logger.Infof("Created container image '%s' for %s", job.Image, job.Name)
    if err = d.client.ContainerStart(ctx, resp.ID, container.StartOptions{}); err != nil {
        return "", err
    }
}
```

Wait for Container and Get Logs

```
d.logger.Infof("Waiting for container image '%s' for %s", job.Image, job.Name)
msg, err := d.waitForContainer(ctx, job, resp.ID)
if err != nil {
    return msg, err
}

d.logger.Infof("Done waiting for container image '%s' for %s", job.Image, job.Name)
// extract the logs before removing the container below
logReader, err := d.client.ContainerLogs(ctx, resp.ID, container.LogsOptions{
    ShowStdout: true,
})
if err != nil {
    return "", err
}
```

```
// Job describes how to execute a CI job.
type Job struct {
    // Name describes the running job; mainly used to name docker containers.
    Name string
    // Image names the image to use to run the job.
    Image string
    // Dockerfile contents.
    // If empty, the image is assumed to exist.
    // If non-empty, the image is built from this Dockerfile.
    Dockerfile string
    // BindDir is the directory to bind to the container's /quickfeed directory.
    BindDir string
    // Env is a list of environment variables to set for the job.
    Env []string
    // Commands is a list of shell commands to run as part of the job.
    Commands []string
}

// Runner contains methods for running user provided code in isolation.
type Runner interface {
    // Run should synchronously execute the described job and return the output.
    Run(context.Context, *Job) (string, error)
}
```

```
var hostConfig *container.HostConfig
if job.BindDir != "" {
    hostConfig = &container.HostConfig{
        Mounts: []mount.Mount{
            {
                Type:    mount.TypeBind,
                Source:  job.BindDir,
                Target:  QuickFeedPath,
            },
        },
    },
}

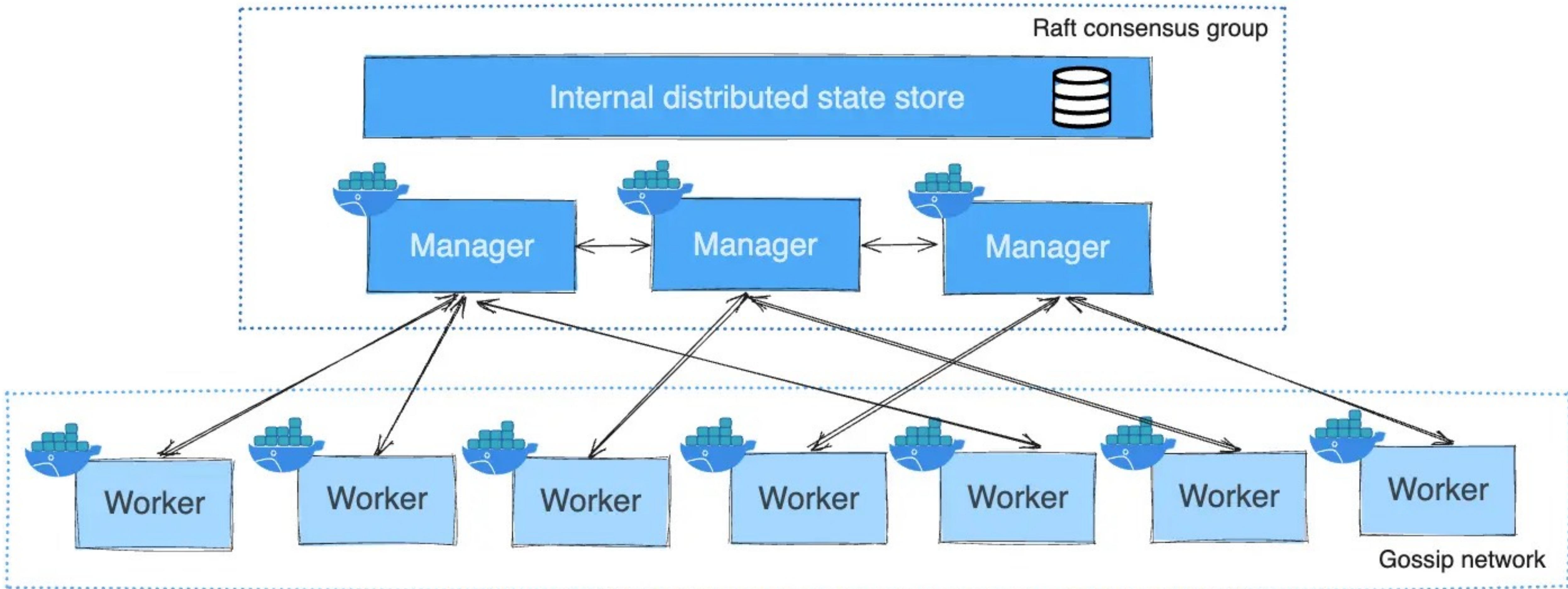
create := func() (container.CreateResponse, error) {
    return d.client.ContainerCreate(ctx, &container.Config{
        Image: job.Image,
        User:   fmt.Sprintf("%d:%d", os.Getuid(), os.Getgid()), // Run the image as the current user, e.g., quickfeed
        Env:    job.Env,                                         // Set default environment variables
        Cmd:    []string{"/bin/bash", "-c", strings.Join(job.Commands, "\n")},
    }, hostConfig, nil, nil, job.Name)
}

resp, err := create()
```

Docker Container Orchestration

Container Orchestration Services

- **Scaling**
 - Automatically scale containers across multiple nodes in a Docker Swarm
- **Load Balancing**
 - Distribute incoming requests evenly across containers within the service
- **Declarative Management**
 - Define desired state of services
 - Docker ensures the correct number of containers are running



Orchestration Components



Questions?